

Plug-in enhancement framework: Breaking through performance bottleneck of pre-trained models for encrypted traffic classification

Chaofan Zheng ^a, Hailong Ma ^{a,b,*}, Yanze Qu ^a, Yiming Jiang ^{a,b}, Wenbo Wang ^{a,*}

^a Information Engineering University, China

^b Ministry of Education of China, Key Laboratory for Cyberspace Security, China

ARTICLE INFO

Keywords:

Encrypted traffic classification
Pre-training
Plug-in enhancement framework
Feature fusion

ABSTRACT

With the proliferation of encryption technologies, identifying network traffic content has become increasingly challenging, rendering malicious communications difficult to detect. Pre-trained models (PTMs) alleviate the constraints of limited labelled data and exhibit significant potential for encrypted traffic classification. However, existing PTMs can only process fixed-length raw byte payloads, facing dual challenges of insufficient feature extraction and input length limitations. Addressing this research gap, we have designed a pluggable enhancement framework. In contrast to conventional feature layer concatenation, our framework resolves spatial mismatches between heterogeneous features through a designed Statistical Feature Semantic Mapping network. It employs an adaptive gated fusion mechanism to achieve incremental enhancement without modifying or disrupting the original PTM weights. The framework's novelty lies in its plug-and-play capability, enabling seamless integration into diverse architectures such as Mamba and Transformer to elevate the classification accuracy of traditional models. Experiments on multiple public datasets show that this framework improves the F1 score by up to 26%, 20%, 55%, and 14% for four mainstream PTMs. The practical significance of this research lies in enabling legacy models with limited performance to rapidly enhance classification accuracy for covert malicious traffic through this plug-in, without requiring time-consuming secondary fine-tuning. Despite these significant improvements, the current framework cannot yet effectively counter complex traffic obfuscation techniques and fails to achieve joint detection of multi-stream correlated behaviours. This study offers novel insights for achieving lightweight performance breakthroughs in real-world intrusion detection systems (IDS). The source code will be released at <https://github.com/slg6/Plug-inEnhancementFramework>.

1. Introduction

With the widespread adoption of Transport Layer Security (TLS) and subsequent encryption technologies, over 95% of internet traffic is now encrypted via Hypertext Transfer Protocol Secure (HTTPS) [1]. Whilst encryption effectively safeguards user privacy and communication security, the complete invisibility of encrypted traffic content presents unprecedented challenges for network traffic classification and potential threat detection. Zscaler's report [2] indicate that, as of 2023, over 85% of cyber threats are delivered via encrypted channels.

Traditional machine learning approaches rely heavily on large volumes of labelled data, yet labelling encrypted traffic incurs substantial costs. To address this challenge, PTM has emerged as a cutting-edge direction for encrypted traffic classification [3]. By performing self-supervised learning on vast amounts of unlabelled traffic, the model extracts deep semantic features, significantly alleviating the difficulty

of labelling malicious samples. However, in practical industrial deployment and real-world applications, existing PTMs for encrypted traffic face severe performance bottlenecks and practical deployment challenges, primarily manifested in the following two aspects:

Firstly, severe information truncation and feature loss occur. Constrained by the computational complexity of deep learning models (particularly the Transformer architecture) [4], existing PTM [5–8] typically only accept fixed-length inputs (such as 128 or 512 bytes). In practical network environments, this necessitates the model discarding the vast majority of packet payloads or flow-level features. This 'perspective limitation due to input constraints' enables models to capture local byte-level semantics while completely losing the ability to perceive global statistical characteristics such as flow duration, average packet interval, and total transmission rate. These features are crucial for identifying threats with unique behavioural patterns, such as Denial of Service (DoS) attacks [9].

* Corresponding authors.

E-mail addresses: slg1233212024@163.com (C. Zheng), longmanclear@163.com (H. Ma), quyanze@foxmail.com (Y. Qu), j8403@163.com (Y. Jiang), wbwangu@outlook.com (W. Wang).

<https://doi.org/10.1016/j.comnet.2026.112122>

Received 26 July 2025; Received in revised form 15 February 2026; Accepted 15 February 2026

Available online 17 February 2026

1389-1286/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Secondly, model updating incurs high costs and lacks flexibility. In practical industrial environments (such as ISP-level firewalls or enterprise perimeter security gateways), traffic distribution fluctuates dramatically with business changes [10,11]. Existing PTMs are typically black-box entities. Integrating new features or optimising for novel tasks often necessitates full fine-tuning of extensive parameters—an approach unacceptable in resource-constrained edge computing or real-time on-line systems requiring continuous updates [12,13]. Consequently, enhancing existing model performance through ‘low-intrusive’ methods has become the core challenge for deploying PTMs in practical industrial scenarios.

In response to the aforementioned challenges, we propose a pluggable augmentation framework. Rather than attempting to construct a more cumbersome PTM, we have designed a lightweight, independently attachable augmentation module. The framework is founded upon the construction of a dynamic feature fusion mechanism, the purpose of which is to achieve efficient fusion of statistical features and semantic features. However, achieving the fusion of statistical and semantic features is challenging. The semantic feature vectors extracted by PTM (e.g., [CLS] tokens) are characterised by their highly non-linear semantic combinations, while the statistical feature vectors extracted manually are parallel across dimensions and lack semantic relationships. Consequently, the fusion of these two types of features poses two significant challenges. Firstly, the issue of semantic mismatch between features must be addressed; secondly, the semantic information within semantic features must be preserved during the fusion process.

To address the first challenge, we designed a semantic extraction network for traffic features based on the traditional Multi-Layer Perceptron (MLP). This architecture enables more efficient extraction of semantic information from statistical features, thereby achieving precise alignment between statistical features and the semantic features extracted by PTM. To address the second challenge, we innovated upon the gated network architecture by proposing a self-attention-scalar gated unit. This incorporates self-attention mechanisms and interactive features. These enhancements not only strengthen semantic feature preservation but also foster feature interactions, achieving a balanced interplay between semantic and statistical features. Consequently, the effectiveness of feature fusion is significantly elevated.

We validated the framework’s effectiveness across ten traffic classification tasks using PTMs from four distinct architectures. Experimental results demonstrate that for each model, the maximum improvement in F1 score exceeds 10%. Furthermore, the framework offers a high degree of flexibility and compatibility. In the context of fine-tuned models, the framework has the capacity to enhance model performance without the necessity of retraining the fine-tuned model parameters. This feature offers significant convenience for rapidly improving model performance in practical applications. We further investigated the framework’s key components, revealing the impact of each element on classification performance.

Our principal contributions are summarised as follows:

1. We propose a universal, pluggable traffic classification augmentation framework. This framework integrates seamlessly into various mainstream PTMs without requiring modifications to model architecture or retraining of model weights.
2. Addressing the heterogeneity of traffic features, we design a semantic mapping network and a gated fusion network to overcome dimensional mismatches and semantic discontinuities between statistical and semantic features.
3. Conducted extensive evaluations across ten traffic classification tasks (involving nine public datasets) and four PTMs. Experiments demonstrate that the framework can boost the F1 score of existing PTM by up to 55%, while maintaining consistent enhancement effects on fine-tuned models.

2. Related work

2.1. Traffic classification

Early research on encrypted traffic classification heavily relied on feature engineering and classical machine learning algorithms. Machine learning-based approaches [14–18] depended on feature extraction engineering informed by expert knowledge. Following the construction of feature vectors, classification was performed using models such as decision trees, random forests, and support vector machines (SVM). Flow-Print [19] proposed a semi-supervised fingerprinting approach, constructing application fingerprints by analysing temporal correlations between target addresses within encrypted traffic. The performance of such methods is highly dependent on the quality of feature selection and domain expertise, whilst manually designed features exhibit limited generalisation capabilities when confronted with emerging applications or protocol variants [20].

With the proliferation of deep learning technologies, research has shifted towards models capable of automatically learning feature representations. Deep learning-based approaches [21–25] learn complex patterns directly from raw data via end-to-end models, circumventing feature engineering. FS-Net [26] utilises traffic behaviour sequences (packet length sequences) to automatically learn discriminative features through a BiGRU-based encoder-decoder architecture, enabling end-to-end classification of encrypted traffic. Hi-Fi [27] proposes a hierarchical deep learning architecture that unifies heterogeneous traffic into fixed-dimensional vector representations through progressive abstraction from packets to flows to traffic traces, significantly enhancing the generalisation capability for fingerprint recognition tasks. However, these deep learning approaches remain fundamentally supervised learning methods, whose performance bottleneck lies in their reliance on large volumes of high-quality annotated data. Annotating network traffic is costly and requires specialised expertise.

2.2. Pre-training models

To alleviate the scarcity of labelled data, drawing upon successful experiences from the natural language processing domain, PTM has been introduced into the field of encrypted traffic classification [3]. Its core concept involves performing self-supervised pre-training on large-scale unlabelled traffic data to learn generalised traffic representations, followed by fine-tuning with a small amount of labelled data to adapt to downstream classification tasks. Research has attempted to directly transfer the BERT architecture [28] to the traffic analysis domain. For instance, ET-BERT [5] proposes a Transformer-based PTM that learns deep contextual representations from large-scale raw traffic bytes through self-supervised learning, adaptable to downstream classification tasks via fine-tuning. TrafficFormer [6] enhances PTM’s understanding of traffic protocol semantics and patterns by introducing sequence direction prediction tailored to traffic characteristics alongside data augmentation strategies, demonstrating strong performance with minimal labelled data. YaTC [7] treats traffic data as images, learning multi-level flow representations through unsupervised pre-training, significantly improving classification data efficiency. Other research, such as NetMamba [8], proposes a PTM based on the modern state space model (Mamba), utilising unidirectional modelling with linear complexity and masked autoencoders to achieve efficient and low-bias encrypted traffic classification.

Despite these notable advancements, PTMs share a common limitation: their relatively rigid model structures primarily rely on learning semantic features from raw bytes or finite-element data. They lack a flexible, standardised mechanism for efficiently integrating artificially constructed, domain-knowledge-rich global statistical features (e.g., flow duration, packet length variance). This limitation in multi-source heterogeneous feature fusion constrains further performance gains. To address this, this paper proposes a pluggable feature enhancement

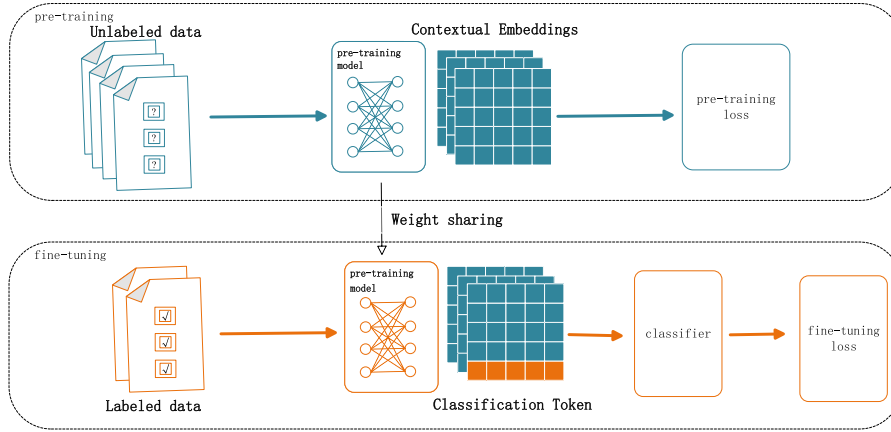


Fig. 1. Pre-training and fine-tuning.

framework. By designing a semantic extraction network and a gated fusion mechanism, it achieves lossless, adaptive fusion of statistical features with pre-trained semantic features. This significantly improves classification performance without modifying the PTM architecture, offering a flexible and scalable enhancement paradigm for encrypted traffic classification.

3. Background and problem description

The prevailing research approach to address the challenge of classifying encrypted traffic involves the utilization of information contained within data packets to train machine learning [16,18,19] or deep learning models [23,25,26]. These models are then capable of discerning between diverse traffic characteristics, thereby facilitating successful classification. Traditional traffic classification models suffer from reliance on high-quality labelled data and poor generalisation capabilities.

However, in contrast to data types that are more closely related to everyday life, such as text, images, and sound, the annotation of traffic data requires staff to not only understand network protocols but also possess professional knowledge to identify specific network scenarios and attack patterns. Furthermore, the presence of malicious traffic can be obscured by the substantial volume of normal traffic, thus complicating the process of manual annotation.

PTMs [28–30] have demonstrated considerable potential in addressing the issue of limited labelled data. Consequently, several studies have introduced pre-training methods into the domain of traffic classification. The pre-training methodology comprises pre-training phase and fine-tuning phase, as illustrated in Fig. 1. During the pre-training stage, PTM acquires general knowledge through the utilization of a substantial volume of unlabelled data. Subsequently, during the fine-tuning stage, it assimilates task-specific knowledge through the analysis of a limited amount of labelled data. Consequently, optimal classification performance can be attained with a limited amount of labelled data, thereby reducing the requirement for additional labelled data and enhancing the model's generalisation capability. However, contemporary pre-training methodologies employ raw data bytes exclusively for classification, thereby disregarding information that lies beyond the payload. Furthermore, due to their fixed input length, pre-training methods are unable to capture the global features of traffic data.

As demonstrated in Fig. 2, PTMs in the domain of traffic analysis encounter a prevalent challenge: they are only capable of processing a limited subset of data stream information and lack the capacity to effectively handle statistical features. A quantitative analysis of information loss caused by data truncation is detailed in Appendix C. The statistical feature vectors extracted from traffic data contain key information such as packet intervals, packet lengths, and packet transmission rates. These

metrics reflect global characteristics of traffic and serve as crucial criteria for traffic classification. However, it should be noted that the initial conception of PTMs was to focus on the extraction of semantic features from raw data bytes, thus neglecting the importance of statistical features. This overreliance on raw data bytes alone has the effect of limiting the classification performance of PTMs when dealing with complex traffic data. Consequently, the fusion of statistical features with the semantic features of PTMs has emerged as a pivotal direction for enhancing the effectiveness of traffic classification.

However, the fusion of semantic vectors with statistical feature vectors gives rise to the challenge of vector mismatch. For the purpose of illustration, consider the ET-Bert [5] model. This model extracts a 768-dimensional semantic vector ([CLS]) from raw bytes for traffic classification. While this semantic vector provides rich semantic information about the input sequence as a whole, its individual dimensions are difficult to interpret because they result from highly nonlinear combinations. Conversely, statistical feature vectors are characterised by the utilization of vectors derived from various clearly defined features (e.g., packet intervals, packet lengths, packet transmission rates, etc.). In order to enhance the classification capabilities of PTMs using statistical feature vectors, it is essential to address the semantic differences between [CLS] and statistical feature vectors, as well as the issue of dimensional mismatch.

In order to address the aforementioned issues, it is proposed that a plug-in framework be designed, with the aim of enabling models to fully use the information contained in statistical features. The implementation of this framework will enhance the classification performance of the models. The term 'plug-in' refers to the fact that this framework can seamlessly integrate and enhance the classification performance of PTMs without modifying their original structure.

4. Framework design

In this section, the motivation and intuition behind the framework design are introduced. The framework's workflow is then outlined, and a detailed introduction to the framework's various components and hyperparameter selection is provided.

4.1. Motivation and intuition

PTMs have been shown to excel at extracting deep semantic features from encrypted payloads; however, their inherent architectural limitations make it difficult to significantly enhance performance by merely improving the pre-training process itself. Therefore, an intuitive and effective approach is to introduce statistical features of the data stream as supplementary information to compensate for the shortcomings of pure

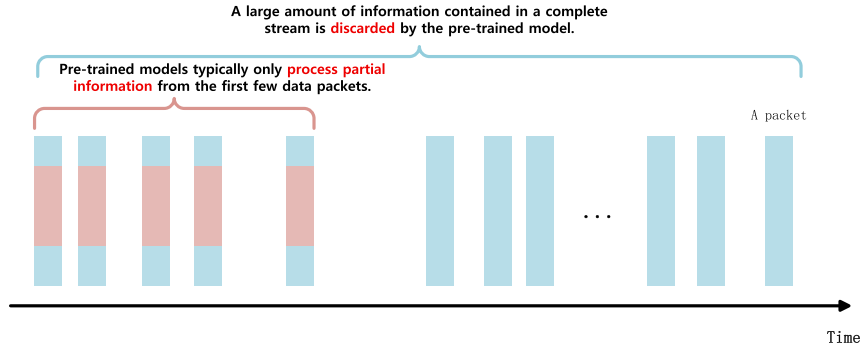


Fig. 2. PTMs can only process a limited amount of information.

semantic modelling. However, a considerable discrepancy has been observed between the statistical features and semantic features extracted by PTMs in terms of semantic space and feature dimensions. The direct fusion of these features without implementing effective processing frequently results in a decline in model classification performance.

First, to effectively fuse extracted statistical features with semantic features obtained via PTM (e.g., [CLS]), the key lies in resolving the disconnect between their representational spaces and semantic meanings. Specifically, this ‘semantic mismatch’ manifests on two levels: Firstly, spatial heterogeneity. Taking Trafficformer and ET-Bert as examples, their [CLS] vectors (768-dimensional) and statistical feature vectors (77-dimensional) exhibit significant dimensional disparities and reside in distinct embedding spaces. Direct vector operations (such as concatenation or weighting) fail to achieve effective fusion due to scale inconsistencies. Secondly, semantic heterogeneity arises because each dimension of the [CLS] vector represents a nonlinear combination of byte sequences, carrying context-dependent deep semantics. Conversely, each dimension of the statistical feature vector explicitly expresses traffic metrics (e.g., mean packet length, variance), with dimensions being independent and lacking contextual correlation. This disparity in representation causes direct concatenation to distort the semantic manifold structure learned by PTM.

To this end, we have designed a lightweight semantic extraction module whose core function is to project two types of features onto a compatible space, ensuring they possess similar semantic information and dimensional scales prior to fusion. Semantic alignment requires no learning of complex contextual dependencies. It merely maps the structured behavioural information of statistical features onto a semantic representation sharing the same dimensionality and scale as the [CLS]. This objective constitutes a feature space transformation rather than complex semantic generation. The parameter scale of a lightweight MLP suffices to cover the mapping requirements, while avoiding the overfitting and semantic redundancy that deep networks may induce.

Second, following feature alignment, the fusion strategy must also satisfy a key constraint: it must not disrupt the inherent semantic structure learned by the PTM. In consideration of the aforementioned constraint, and in light of the successful experiences that have been accumulated in the domain of multi-modal feature fusion [31–33], it has been determined that the adoption of a gating mechanism is to be undertaken in order to facilitate the achievement of adaptive vector fusion. The gating network is capable of dynamically balancing the importance of the two types of features according to task requirements, thereby effectively preserving the integrity of the pre-trained semantics.

4.2. Framework overview

The framework is primarily divided into three parts: extracting statistical features from data streams, extracting semantic information from statistical features, and dynamic fusion. Fig. 3 shows the overall diagram of the framework.

The framework first performs data cleansing on raw traffic (processing retransmitted packets) and divides it into independent network flows based on quintuple criteria. Subsequently, the feature extraction module captures the macro-behavioural patterns of these flows throughout their lifecycle, generating statistical feature vectors ([SFV]). This module compensates for the loss of global contextual information (such as flow duration and packet interval distribution) inherent in PTM due to its input length constraints, thereby providing macro-behavioural features for classification. As [SFV] constitutes numerical features in physical space, while PTM extracts residual vectors ([CLS]) from a highly abstract semantic space, a significant representational mismatch exists between the two. To address this, we designed the Statistical Feature Semantic Extraction Module (SFSEM). This module leverages the non-linear mapping capability of a Multi-Layer Perceptron (MLP) structure to project the [SFV] onto a high-dimensional semantic space isomorphic to PTM, generating a Statistical Feature Semantic Vector ([SFSV]). This process achieves ‘dimensional alignment’ and ‘contextual alignment’ between statistical metrics and byte-level payload semantics, laying the groundwork for subsequent deep fusion. To introduce statistical guidance while preserving pre-trained knowledge integrity, the Gating Fusion Network (GATING) receives vectors from two distinct modalities ([SFSV] and [CLS]). This module employs a self-attention mechanism to compute interaction weights, dynamically adjusting the fusion ratio between the two. Its interactive logic operates as follows: when traffic payloads exhibit semantic poverty due to high encryption levels, the gating mechanism automatically increases the weight of the [SFSV]; conversely, it prioritises preserving the deep payload information extracted by the [CLS]. Finally, the fused composite vector is input into the Classification Head, which employs a Softmax function to predict the probability distribution of traffic belonging to each label. The label with the highest probability is selected as the final classification decision.

4.3. Statistical feature extraction

Statistical feature extraction can be defined as the process of generating low-dimensional feature vectors by calculating the distribution statistics (such as mean, variance, and skewness) of numerical fields from network data streams using a sliding window. A standard for extracting network flow statistical features has been provided, incorporating a total of 77 dimensions (see Appendix A). While this standard does not guarantee optimal experimental performance across all datasets, it enables significant performance improvements for PTMs within the framework. The framework itself imposes no strict restrictions on the dimension of the statistical feature vector. Users are able to select the features to extract based on the specific characteristics of the dataset, thereby better adapting to the traffic classification requirements of different scenarios.

Improvements have been made to the CICFlowMeter [34] project (CFM), with the objective of extracting statistical features of network flows. When employing CFM for the purpose of statistical feature

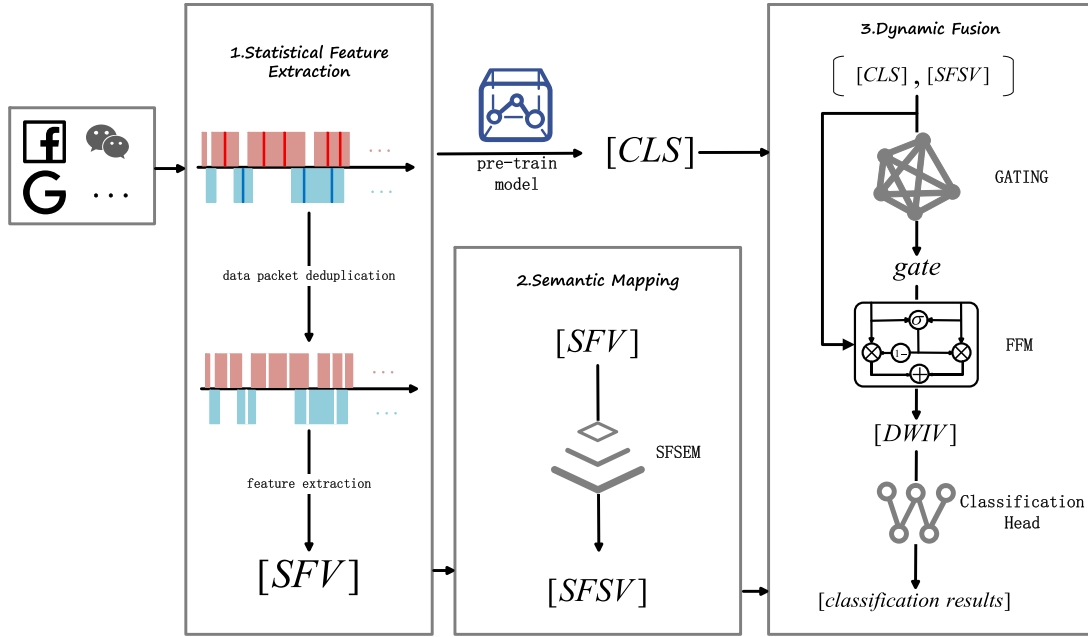


Fig. 3. Overall architecture: 1) Process raw traffic data and extract statistical feature vectors. 2) Employ the Statistical Feature Semantic Extraction Module to map the statistical feature vector into semantic space, yielding the statistical feature semantic vector ([SFSV]). 3) Utilise the multi-head self-attention gating mechanism (GATING) and Feature Fusion Module (FFM) to fuse the [CLS] with [SFSV], producing the Dynamic Weighted Interaction Vector ([DWIV]). This is then subjected to classification prediction via a classification head.

Table 1
Statistics on retransmitted packets in the USTC-TFC2016 classification.

Category	Total number of data packets	Proportion of retransmitted packets
Cridex	461548	13.03%
Geod	250000	26.37%
Miuref	88560	13.66%
Neris	499218	19.45%
Virut	440625	30.79%
Zeus	93141	20.27%

extraction, it was discovered that the retransmission of certain packets within the data stream resulted in CFM erroneously disconnecting the data stream at the point of retransmission, thereby identifying it as two distinct streams. Furthermore, a substantial number of retransmitted packets have been shown to interfere with the accuracy of network flow statistical features. However, retransmission is a common phenomenon in real network flows. As demonstrated in Table 1, an analysis was conducted on the six categories with the highest proportions of retransmitted packets in the USTC-TFC2016 [35], revealing that certain categories exhibited retransmission rates of up to 30% or more. Consequently, the direct utilization of raw traffic data for statistical feature extraction can yield a substantial number of statistical errors, thereby impacting classification performance.

To address the aforementioned issues, we perform data cleansing prior to statistical feature extraction. Specifically, to resolve flow segmentation errors and statistical feature calculation biases caused by retransmitted packets, we first reconstruct and uniquely identify network flows using the quintuple (source IP, destination IP, source port, destination port, transport layer protocol) – ensuring all packets belonging to the same data stream are correctly grouped together. During packet filtering, we employ the tshark [36] to validate packets: when two packets share identical quintuplets, sequence numbers, and payload lengths, they are identified as duplicate retransmissions and discarded to guarantee the authenticity of statistical features. Following data cleansing, we employ CFM to sequentially extract the [SFV] for each reconstructed

network flow, adhering to the 77-dimensional feature selection criteria defined in Appendix A.

We selected the 77-dimensional feature set based on CICFlowMeter to establish a universal, reproducible statistical feature benchmark. This set encompasses statistical information across multiple dimensions, including flow duration, packet length distribution, packet interval, and flow directionality. It has been demonstrated in multiple studies to possess discriminative capability for traffic classification [37]. To validate the selected features and the framework's robustness, we conducted the following analyses (see Section 5.4): Feature importance analysis indicates that the ranking of statistical features' significance is scenario-dependent, with key features for traffic classification varying significantly across different network scenarios; Feature subset experiments further demonstrate that even when employing only the top 20, 40, or 60 dimensional feature subsets, the proposed framework effectively enhances the model's classification performance, indicating robust resilience to feature selection. In practical deployments, users may adjust the feature set according to specific scenarios, highlighting the framework's strong scalability.

4.4. Semantic mapping

The purpose of semantic mapping is to project manually extracted feature vectors onto a semantic space, thereby resolving the semantic and dimensional mismatch between statistical feature vectors and semantic feature vectors. Our in-depth analysis of extracted network traffic statistics reveals the following characteristics: numerous dimensions within the vectors contain zero values; the numerical ranges across dimensions exhibit sparse distributions and predominantly non-negative values. Given these characteristics, it is essential to circumvent the inherent limitations of conventional activation functions when applied to such features.

Our proposed Semantic Feature Space Mapping Network (SFSEM) employs a lightweight two-layer architecture: the input layer dimension aligns with the [SFV], the number of hidden layer neurons n is adaptively adjusted based on the [CLS] dimension of the model (see Section 4.6), and the output layer dimension matches the [CLS]. To

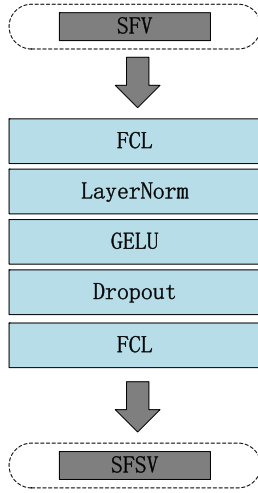


Fig. 4. Semantic mapping network structure.

ensure training stability, a LayerNorm layer is introduced before the activation function. To suppress model overfitting, a Dropout layer is added on the output side (with a dropout rate of 0.1).

Given the sparse non-negative nature of network traffic statistical features, comparative experiments validated the suitability of several mainstream activation functions, including ReLU [38], LeakyReLU [39], Softplus, Sigmoid, Mish [40], tanh, and GeLU [41]. Experimental results indicate that GeLU exhibits superior training stability, demonstrating significantly enhanced performance compared to other activation functions in scenarios such as multi-task traffic forecasting and anomaly detection (see Section 5.4).

Fig. 4 illustrates the specific architecture of the semantic mapping network. For the GeLU activation function, incorporating a normalisation layer aids in stabilising and accelerating the learning process [42]. Consequently, we have added a normalisation layer preceding the activation function. Furthermore, we have introduced a dropout layer to mitigate the risk of model overfitting.

4.5. Dynamic fusion

The specific structure of the dynamic fusion network is illustrated in Fig. 5(a). The vector obtained by concatenating [CLS] and [SFSV] is then used as input, and the final output is the classification result of the network flow. In order to ascertain the complex semantic vector correlations between [CLS] and [SFSV], a special multi-head self-attention gating mechanism (GATING) was designed. In contradistinction to conventional gating networks, the gating mechanism under scrutiny in this study extracts high-order correlation features through the utilization of attention, subsequently generating gating signals. This constitutes a two-stage optimisation process of feature enhancement and dynamic filtering. Furthermore, the conventional single-layer linear transformation, which is frequently employed in gating, is substituted with an MLP to augment non-linear fitting capabilities, thereby facilitating the learning of more complex gating rules. The output layer of the multi-layer perceptron (MLP) in GATING has a dimension of 1, and a sigmoid function is added after the output layer. This indicates that the dimension of the gate vector is 1, and its elements range from 0 to 1, i.e., $gate = \{[x], x \in (0, 1)\}$.

As demonstrated in Fig. 5(b), the Feature Fusion Module (FFM) employs the gating weights g , which are generated by GATING, and the vector formed by concatenating [CLS] and [SFSV] as inputs, thereby yielding a Dynamic Weighted Interaction Vector (DWIV). Since g is conceptualised as a one-dimensional vector, the framework is capable of performing feature fusion from the standpoint of the two feature sources (i.e. [CLS] and [SFSV]). The internal semantic relationship be-

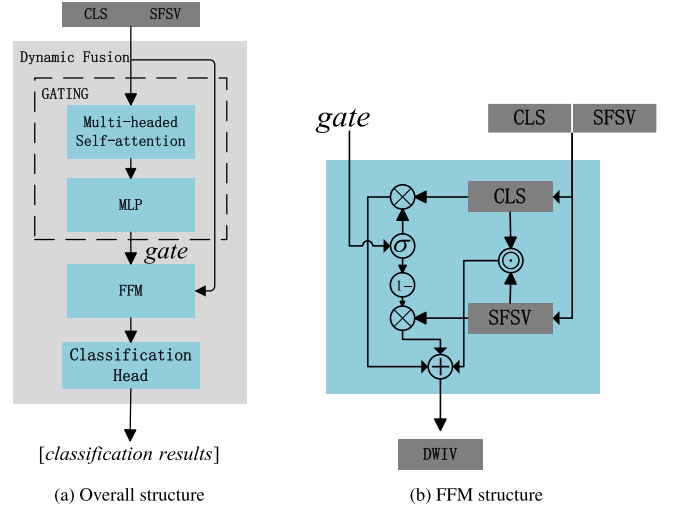


Fig. 5. Dynamic fusion network architecture.

tween [CLS] and [SFSV] is only learned through the semantic extraction module, ensuring that the semantic features of [CLS] and [SFSV] are not compromised. Concurrently, since g is dynamically generated gating weights, setting its dimension to one-dimensional can reduce model complexity and effectively prevent overfitting. Finally, the Hadamard product of the two vectors is added to the fusion vector to enhance feature interaction and strengthen the model's expressive power. In summary, the calculation formula for [DWIV] is as follows:

$$DWIV = gate \times CLS + (1 - gate) \times SFSV + \alpha CLS \odot SFSV \quad (1)$$

Herein, $gate$ denotes the dynamically generated gating weight by FFM, $\times$ represents scalar-vector multiplication (where each element of the scalar is multiplied by each corresponding element of the vector), and $\odot$ denotes the Hadamard product, signifying element-wise multiplication of two vectors of identical dimensions. α , serving as the coefficient for interaction features, is a hyperparameter whose value is obtained through empirical search. The parameter tuning results will be presented in Section 4.6.

4.6. Hyperparameter tuning

Prior to conducting the experiment, it is necessary to ascertain the hyperparameter values of the framework. These comprise the number of hidden layer neurons, n , in the statistical feature semantic extraction network; the coefficient, α , of the interaction features in feature fusion; and the number of heads, m , in the multi-head self-attention in GATING. The selection of n should be based on the principle that the number of hidden layer neurons should be between the number of input layer neurons and the number of output layer neurons. As previously stated, the selection of 77-dimensional statistical features was made, thereby ensuring the dimension of the input layer is fixed at 77. The [CLS] dimension for ET-Bert [5] and TrafficFormer [6] is 768, while for YaTC [7] and NetMamba [8] it is 192. The selection of α is based on the principle that the interaction features and the gated fusion vector should be on the same order of magnitude, so α is centred around 1. For general classification tasks, the number of self-attention heads typically does not exceed 8. Considering the above factors, the search space for each hyperparameter was obtained, as shown in Table 2.

To determine the optimal hyperparameter combination, we conducted a grid search on three representative datasets, namely USTC-TFC, Android and Tor. Each dataset was split into training, validation and test sets at a ratio of 8:1:1. During training, we trained the model for 100 epochs on the training set and calculated the macro-averaged F1-score on the test set at the end of each epoch. We then selected the model corresponding to the epoch with the highest F1-score on the test set for the

Table 2
Framework hyperparameter tuning results.

Model	Hyperparameters	Search space	Final value
ET-Bert [5]	n	[128, 256, 512]	256
	α	[0.2, 0.5, 1, 2, 5]	1
	m	[2, 4, 6, 8]	4
TrafficFormer [6]	n	[128, 256, 512]	256
	α	[0.2, 0.5, 1, 2, 5]	1
	m	[2, 4, 6, 8]	4
YaTC [7]	n	[100, 128, 156]	128
	α	[0.2, 0.5, 1, 2, 5]	1
	m	[2, 4, 6, 8]	4
NetMamba [8]	n	[100, 128, 156]	156
	α	[0.2, 0.5, 1, 2, 5]	1
	m	[2, 4, 6, 8]	4

Table 3
Test model.

Model	Architecture	Total bytes processed	[CLS] dimension
ET-Bert	Transformer	128	768
YaTC	ViT	1600	192
TrafficFormer	TrafficFormer	320	768
NetMamba	Mamba	1600	192

final evaluation on the validation set. The hyperparameter combination yielding the highest F1-score on the validation set was designated as the final configuration, and the resulting experimental results are presented in Table 2. Specifically, the value of n falls exactly between the input and output dimensions; either an excessively large or small value of n leads to a degradation in the semantic extraction capability of the model. α exerts a relatively stable influence on model performance, and a value of 1 achieves the optimal or near-optimal balance on most tasks, indicating that the interactive features and gating fusion features realize effective complementarity at this point. Although increasing m improves the model capacity, bigger is not always better. Experimental results demonstrate that m values of 4 and 6 achieve comprehensively better performance than 2 and 8, as they strike a favorable balance between model performance and computational overhead. Thus, m is uniformly set to 4 in the final configuration.

It should be noted that the aforementioned hyperparameters constitute a set of robust configurations obtained through experimental verification on the selected datasets, which is designed to achieve universal performance improvement of the framework across multiple tasks, rather than to guarantee absolute optimality on all datasets. In practical applications, users can perform fine-tuning according to specific data characteristics. Nevertheless, the default values adopted in this paper have demonstrated remarkable enhancement effects and favorable generalization ability.

5. Evaluation

In this section, an evaluation of the framework is conducted from multiple perspectives. Firstly, the experimental setup is described and the datasets used are introduced. In the subsequent stage of the analysis, the classification performance of the PTM will be compared before and after the utilization of the framework in order to verify the effectiveness of the latter. Furthermore, an analysis is conducted to determine the impact of implementing the framework on a PTM following fine-tuning. The final stage of the research process involves the exploration of the impact of the various components of the framework on overall performance.

5.1. Experiment setup

Test model. As demonstrated in Table 3, in order to substantiate the universality of the framework, an evaluation of its effectiveness was conducted on four distinct PTMs.

Test dataset. As shown in Table 4, we set up ten classification tasks on eight public datasets. Among them, CIC-IOT [43], IDI [44], and ISD [45] are traffic data generated by IoT devices, aiming to test the model's classification ability for IoT device traffic. Service and APP are classification tasks in the ISCX-VPN [46] dataset, respectively categorised by service and software type. Tor is a classification task in the ISCX-Tor [47] dataset categorised by software type. Service, app, and tor are used to test the model's classification performance for channel traffic and anonymous network traffic. UAV [48] consists of traffic data generated by different online behaviours (e.g., video, interaction, idle), aiming to test the model's classification capability for user behaviour. Android [49] is a dataset composed of traffic generated by multiple adware, malware, and normal applications. USTC-TFC [35] is a dataset composed of traffic generated by 10 benign software and 10 malicious software. Our research found that the timestamps of benign software traffic in USTC-TFC were manually modified, differing significantly from malicious software traffic. This modification may have improved the classification performance of statistical features on this dataset. As a result, we extracted all malicious software traffic from the dataset to form a new dataset, ustcMal. Android, USTC-TFC, and ustcMal are used to test the model's classification performance for benign and malicious traffic.

Baseline Methods. To comprehensively evaluate the framework's performance enhancement, comparative experiments were conducted using three representative approaches: the classical semi-supervised machine learning method FlowPrint [19], which generates application fingerprints based on time-correlated features of network traffic destination attributes to classify encrypted traffic; the end-to-end deep learning method FS-Net [26], employing a multi-layer bidirectional GRU encoder-decoder architecture with a reconstruction mechanism to autonomously learn discriminative features from raw flow sequences; the input-agnostic hierarchical deep learning framework Hi-Fi [27], which captures cross-layer spatio-temporal correlations through packet-flow-trace tri-level feature abstraction to adapt to heterogeneous traffic inputs.

Training settings. The training of each model was conducted utilising the PyTorch 2.0.1 framework. The training of ET-Bert, Trafficformer and YaTC was executed through the utilization of CUDA 11.5, while NetMamba necessitated CUDA 11.6 or a more recent version, consequently prompting the adoption of CUDA 11.7 for the training process. The batch size for training all models was set to 16, and the Adamw [50] optimisation algorithm was used for optimisation. Furthermore, the default training parameters for each pre-trained model were retained during the fine-tuning process, as illustrated in Table 5. We will involve a comparison of the classification performance both prior to and following the implementation of the framework, with the objective of evaluating the impact of the framework on the enhancement of the performance of pre-trained models.

Data preprocessing. Initially, each dataset is divided into independent network flows. For categories comprising more than 500 flows, 500 flows are retained. Subsequently, each dataset is divided into training, validation, and test sets in an 8:1:1 ratio.

Evaluation metrics. The performance of the framework is evaluated using four classic metrics: accuracy (AC), precision (PR), recall (RC), and F1 score (F1). In order to circumvent the issue of result bias, which is precipitated by an imbalance in the data type, the macro-average is used to calculate the average values of AC, PR, RC, and F1 for each category. This is due to the multi-class classification problem. To ensure the reliability of experimental results, each experiment employed five distinct random seeds to partition the dataset. The mean and variance of the five independent experimental results were calculated and used as the final metrics for comparative analysis.

5.2. Experimental results

IOT device datasets. Experimental results across the CIC-IoT, IDI, and ISD IoT datasets reveal the framework's differentiated value across

Table 4
Test dataset.

Datasets	Task Description	Class Number
CIC-IoT	Identification of IoT device behaviour	5
IDI	IoT device identification	31
ISD	Identification of IoT botnets	2
Service	Identification of VPN encryption service types	6
APP	Identification of VPN encryption software types	14
Tor	Tor network fingerprinting	5
UAV	User behaviour identification	5
Android	Distinguishing between advertising software, malware, and general software	3
USTC-TFC	Malware detection	20
ustcMal	Malware classification	10

Table 5
Training settings.

Model	Initial learning rate	Epochs*	Input package scope	Field extraction position [†]
ET-Bert	6e-5	100	The first 5	After the 78th byte
TrafficFormer	6e-5	100	The first 5	The 28th to 92nd bytes
YaTC	2e-3	200	The first 5	Header: 0–80 Payload: 0–240
NetMamba	2e-3	120	The first 5	Header: 0–80 Payload: 0–240

*Select the best-performing model from all training rounds for comparison.

[†]The bytes referred to here are the bytes reserved for each data packet, with any insufficient bytes filled with 0.

Table 6
IoT device datasets classification results.

Dataset	CIC-IoT				IDI				ISD			
	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
FlowPrint	0.7147 (±0.021)	0.8105 (±0.005)	0.3710 (±0.017)	0.4709 (±0.017)	0.5580 (±0.038)	0.8306 (±0.024)	0.3813 (±0.035)	0.4635 (±0.034)	0.7111 (±0.039)	0.6568 (±0.005)	0.4772 (±0.024)	0.5482 (±0.017)
FS-Net	0.8516 (±0.014)	0.8471 (±0.014)	0.8344 (±0.013)	0.8376 (±0.013)	0.6487 (±0.055)	0.6123 (±0.052)	0.5951 (±0.051)	0.5818 (±0.050)	0.9219 (±0.013)	0.9204 (±0.013)	0.9204 (±0.015)	0.9204 (±0.013)
Hi-Fi	0.6922 (±0.021)	0.7094 (±0.018)	0.6922 (±0.021)	0.6898 (±0.027)	0.5080 (±0.013)	0.5438 (±0.021)	0.5305 (±0.016)	0.4995 (±0.014)	0.9469 (±0.018)	0.9495 (±0.018)	0.9469 (±0.018)	0.9469 (±0.018)
ET-Bert	0.7525 (±0.051)	0.7516 (±0.054)	0.7442 (±0.051)	0.7435 (±0.053)	0.8232 (±0.035)	0.8649 (±0.023)	0.8551 (±0.021)	0.8538 (±0.019)	0.9464 (±0.020)	0.9476 (±0.021)	0.9440 (±0.022)	0.9449 (±0.021)
ET-Bert ef ^a	0.7999 (±0.037)	0.7975 (±0.039)	0.7949 (±0.036)	0.7943 (±0.037)	0.8215 (±0.029)	0.8687 (±0.019)	0.8560 (±0.016)	0.8549 (±0.015)	0.9539 (±0.020)	0.9526 (±0.022)	0.9545 (±0.018)	0.9530 (±0.020)
TrafficFormer	0.7648 (±0.016)	0.7704 (±0.019)	0.7648 (±0.016)	0.7630 (±0.013)	0.9095 (±0.010)	0.9328 (±0.006)	0.9310 (±0.007)	0.9302 (±0.006)	0.9720 (±0.013)	0.9737 (±0.011)	0.9720 (±0.013)	0.9720 (±0.013)
TrafficFormer ef	0.8304 (±0.033)	0.8398 (±0.027)	0.8304 (±0.033)	0.8304 (±0.032)	0.9226 (±0.013)	0.9429 (±0.008)	0.9407 (±0.010)	0.9403 (±0.010)	0.9780 (±0.013)	0.9786 (±0.013)	0.9780 (±0.013)	0.9780 (±0.013)
YaTC	0.8584 (±0.079)	0.8585 (±0.079)	0.8584 (±0.079)	0.8568 (±0.080)	0.9214 (±0.000)	0.9228 (±0.012)	0.8992 (±0.009)	0.9065 (±0.008)	0.9700 (±0.000)	0.9705 (±0.001)	0.9700 (±0.000)	0.9700 (±0.000)
YaTC ef	0.8792 (±0.069)	0.8809 (±0.068)	0.8792 (±0.069)	0.8790 (±0.069)	0.9228 (±0.003)	0.9356 (±0.005)	0.9067 (±0.008)	0.9138 (±0.009)	0.9740 (±0.005)	0.9750 (±0.005)	0.9740 (±0.005)	0.9740 (±0.005)
NetMamba	0.8344 (±0.094)	0.8358 (±0.093)	0.8344 (±0.094)	0.8334 (±0.095)	0.8632 (±0.013)	0.8912 (±0.010)	0.8657 (±0.021)	0.8738 (±0.016)	0.9560 (±0.005)	0.9568 (±0.005)	0.9560 (±0.005)	0.9560 (±0.005)
NetMamba ef	0.7320 (±0.307)	0.8341 (±0.107)	0.8620 (±0.078)	0.8872 (±0.094)	0.8612 (±0.025)	0.8998 (±0.020)	0.8727 (±0.023)	0.8817 (±0.021)	0.9700 (±0.010)	0.9703 (±0.010)	0.9700 (±0.010)	0.9700 (±0.010)

^a“ef” indicates that the model has been enhanced using this framework.

diverse IoT application scenarios (see Table 6). The most significant performance improvement was observed in the CIC-IoT dataset (device behaviour identification), where TrafficFormer achieved an F1 score increase exceeding 8.8%. This improvement holds significant practical importance: in smart home or industrial IoT environments, accurately identifying device behaviour (such as sensors being hijacked for DDoS attacks) is crucial for security protection. Traffic in this dataset originates from diverse application types (video streaming, file transfers, control commands), exhibiting markedly distinct statistical characteristics (e.g., packet length distribution, transmission rates). Our framework effectively addresses the information gap inherent in pure deep learning approaches when handling encrypted IoT traffic, where protocol headers conceal critical details, by integrating these side-channel features.

In contrast, improvements in IDI (device type identification) are limited. This stems from the high similarity in configuration behaviours across different brands of devices within home router setups, resulting in inherently low discriminative power of statistical features. Consequently, the model relies more heavily on payload content features for classification. This suggests that in scenarios with high device fingerprint similarity, optimising semantic feature extraction should take precedence over reliance on statistical feature enhancement. The performance of ISD (botnet detection) has approached saturation ($F1 > 0.95$),

with inherently limited scope for improvement; consequently, gains in F1 scores are generally constrained.

Tor and VPN encrypted datasets. As shown in Table 7, within the encrypted datasets of VPN and Tor, our framework demonstrates significant effectiveness on the Tor dataset. It achieves over 20% improvement in F1 scores for both the ET-Bert and Trafficformer models, approximately 55% improvement for YaTC, and around 15% improvement for Netmamba.

The framework achieves a significant breakthrough in Tor network fingerprinting tasks, yielding results of substantial practical value for network surveillance and threat intelligence analysis. Tor traffic undergoes multi-layered encryption, severely diluting the semantic information within its byte sequences, rendering traditional methods nearly incapable of distinguishing between different anonymising applications. However, our framework successfully identifies distinct application behavioural fingerprints by capturing statistical features such as traffic burst patterns, stream duration, and packet arrival intervals. For instance, web browsing traffic from the Tor Browser exhibits a pronounced ‘request-response’ intermittent pattern, whereas file transfer applications demonstrate sustained high throughput. These behavioural side channels, obscured by multiple layers of encryption in the raw byte stream, remain discernible through statistical characteristics.

Table 7
Tor and VPN encrypted datasets classification results.

Dataset	Service				APP				Tor			
	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
FlowPrint	0.3379 (±0.023)	0.7677 (±0.116)	0.3539 (±0.024)	0.4613 (±0.040)	0.3362 (±0.088)	0.6923 (±0.067)	0.2707 (±0.054)	0.3508 (±0.064)	0.2026 (±0.057)	0.3820 (±0.083)	0.1134 (±0.051)	0.1646 (±0.060)
FS-Net	0.9292 (±0.079)	0.9301 (±0.079)	0.9357 (±0.080)	0.9328 (±0.080)	0.6920 (±0.059)	0.5421 (±0.046)	0.5323 (±0.045)	0.5272 (±0.045)	0.8442 (±0.072)	0.5834 (±0.050)	0.5961 (±0.051)	0.5708 (±0.049)
Hi-Fi	0.6855 (±0.060)	0.6854 (±0.059)	0.7154 (±0.057)	0.6772 (±0.054)	0.5913 (±0.031)	0.5845 (±0.033)	0.5333 (±0.048)	0.5146 (±0.035)	0.6000 (±0.084)	0.4376 (±0.053)	0.4417 (±0.137)	0.3946 (±0.071)
ET-Bert	0.9347 (±0.015)	0.9378 (±0.016)	0.9375 (±0.014)	0.9365 (±0.015)	0.7500 (±0.038)	0.5973 (±0.037)	0.5861 (±0.024)	0.5836 (±0.031)	0.8780 (±0.035)	0.5729 (±0.135)	0.5391 (±0.123)	0.4969 (±0.125)
ET-Bert ef*	0.9399 (±0.022)	0.9442 (±0.018)	0.9417 (±0.019)	0.9419 (±0.019)	0.7729 (±0.047)	0.6037 (±0.057)	0.5790 (±0.055)	0.5811 (±0.053)	0.8916 (±0.033)	0.6230 (±0.145)	0.6473 (±0.129)	0.6237 (±0.130)
TrafficFormer	0.9532 (±0.020)	0.9566 (±0.015)	0.9530 (±0.021)	0.9539 (±0.018)	0.8619 (±0.040)	0.6922 (±0.037)	0.6803 (±0.051)	0.6774 (±0.050)	0.8458 (±0.038)	0.3927 (±0.112)	0.4428 (±0.121)	0.4149 (±0.103)
TrafficFormer ef	0.9482 (±0.019)	0.9498 (±0.019)	0.9493 (±0.018)	0.9485 (±0.019)	0.8559 (±0.022)	0.6663 (±0.026)	0.6578 (±0.032)	0.6552 (±0.025)	0.8625 (±0.043)	0.4946 (±0.098)	0.4998 (±0.127)	0.5029 (±0.108)
YaTC	0.9700 (±0.003)	0.9723 (±0.003)	0.9700 (±0.003)	0.9707 (±0.003)	0.9008 (±0.001)	0.7801 (±0.022)	0.7732 (±0.010)	0.7739 (±0.015)	0.8104 (±0.093)	0.4294 (±0.168)	0.4572 (±0.123)	0.4403 (±0.144)
YaTC ef	0.9707 (±0.007)	0.9730 (±0.007)	0.9710 (±0.007)	0.9714 (±0.007)	0.8987 (±0.012)	0.7828 (±0.058)	0.7618 (±0.025)	0.7612 (±0.030)	0.8961 (±0.023)	0.8999 (±0.057)	0.6312 (±0.079)	0.6840 (±0.067)
NetMamba	0.9664 (±0.004)	0.9678 (±0.005)	0.9666 (±0.004)	0.9665 (±0.004)	0.8731 (±0.012)	0.7651 (±0.047)	0.7399 (±0.029)	0.7399 (±0.030)	0.8701 (±0.013)	0.5522 (±0.163)	0.4772 (±0.064)	0.4765 (±0.091)
NetMamba ef	0.9671 (±0.006)	0.9688 (±0.008)	0.9673 (±0.006)	0.9676 (±0.007)	0.8853 (±0.012)	0.7888 (±0.039)	0.7640 (±0.038)	0.7632 (±0.041)	0.8753 (±0.022)	0.6397 (±0.205)	0.5280 (±0.109)	0.5450 (±0.128)

*“ef” indicates that the model has been enhanced using this framework.

Table 8
User Activities classification result.

Dataset	UAV			
	AC	PR	RC	F1
FlowPrint	0.4331 (±0.055)	0.8170 (±0.012)	0.3337 (±0.041)	0.4375 (±0.050)
FS-Net	0.7203 (±0.061)	0.5935 (±0.051)	0.6263 (±0.053)	0.6041 (±0.051)
Hi-Fi	0.6416 (±0.034)	0.6605 (±0.031)	0.6416 (±0.034)	0.6411 (±0.030)
ET-Bert	0.6591 (±0.052)	0.6686 (±0.049)	0.6576 (±0.051)	0.6553 (±0.040)
ET-Bert ef*	0.7218 (±0.044)	0.7347 (±0.026)	0.7180 (±0.038)	0.7164 (±0.033)
TrafficFormer	0.8562 (±0.035)	0.8311 (±0.035)	0.8317 (±0.045)	0.8286 (±0.040)
TrafficFormer ef	0.8710 (±0.027)	0.8553 (±0.030)	0.8507 (±0.029)	0.8493 (±0.030)
YaTC	0.7944 (±0.010)	0.7981 (±0.010)	0.7944 (±0.010)	0.7937 (±0.009)
YaTC ef	0.8528 (±0.013)	0.8580 (±0.012)	0.8528 (±0.013)	0.8536 (±0.012)
NetMamba	0.7816 (±0.010)	0.7831 (±0.012)	0.7816 (±0.010)	0.7813 (±0.011)
NetMamba ef	0.8400 (±0.008)	0.8473 (±0.009)	0.8400 (±0.008)	0.8413 (±0.008)

*“ef” indicates that the model has been enhanced using this framework.

User Activities (UAV). As demonstrated in Table 8, on the UAV dataset, the efficacy of our framework was demonstrated by the enhancement of the classification performance of all models. Among the models examined, the ET-Bert model demonstrated the most substantial enhancement, exhibiting an F1 score increase exceeding 10%.

User behaviour on the UAV dataset is characterised by significant variability, resulting in inherent differences in the content of traffic streams. Consequently, the classification performance of pre-trained models is not significantly suboptimal. Furthermore, traffic generated by different user behaviours exhibits distinct statistical differences, which are difficult to capture solely through traffic payload analysis. Therefore, supplementing the model’s statistical characteristics through the framework further enhances classification performance, significantly improving the granularity of behavioural recognition. This holds direct application value for corporate compliance auditing and internal threat detection. In practical deployment, this enhancement enables security operations centres to identify unauthorised data exfiltration without decrypting HTTPS traffic.

Malware dataset. As demonstrated in Table 9, the framework has achieved substantial enhancements in model performance on the Android, USTC-TFC, and ustcMal datasets. The framework was found to have a significant impact on the performance of the TrafficFormer model, with the F1 score being increased by more than 10% on the Android dataset. Furthermore, the F1 score of the ET-Bert model was enhanced by over 5% on the ustcMal dataset.

The proposed framework demonstrates a marked enhancement in the performance of the model on the Android dataset; however, the enhancement is comparatively less pronounced on the USTC-TFC and ustcMal datasets. This discrepancy can be attributed to the fact that

the Android dataset differentiates between various types of software, whereas the USTC-TFC and ustcMal datasets are designed to discern differences within the same software type. The statistical characteristics that differentiate between the various types of software are more pronounced, resulting in a more significant enhancement in model performance through the framework. This reveals a crucial deployment insight: when classifying applications with highly similar functionalities, the discriminative power of statistical features diminishes marginally. In such cases, dynamic behavioural analysis or more granular semantic features should be incorporated.

In cybersecurity applications, misclassification may lead to two primary risks: false negatives (where attack traffic is misjudged as normal) and false positives (where normal traffic is misjudged as an attack). For instance, in Android malware datasets, misclassifying malicious traffic as normal may result in device infection; whereas in Tor datasets, false positives could disrupt legitimate communications for anonymous users. By incorporating statistical features, this framework substantially reduces false positive rates across both Tor and malware datasets, thereby enhancing system reliability. Nevertheless, classification risks remain unresolved, necessitating systematic optimisation through engineering practices during practical deployment.

5.3. Performance improvement for fine-tuned models

The framework is distinguished by its high degree of flexibility, which allows it to be integrated as a plugin into PTMs that have already been fine-tuned. In the absence of retraining, the framework has the capacity to enhance the classification performance of the model. In this section, the model is first refined through the utilization of conventional methodologies, that is to say, training in the absence of the framework. Subsequently, the trained model is integrated with the framework, and the framework parameters are trained, whilst the fine-tuned model parameters are frozen. Three distinct types of datasets were selected for the study: CIC-IOT, UAV, and ustcMal, and conducted comparative experiments on all models.

As demonstrated in Table 10, the framework has the capacity to enhance the classification performance of PTMs without altering the parameters of the fine-tuned model. In particular, the classification performance of the ET-Bert model on the UAV and ustcMal datasets has been enhanced to a greater extent than 10%. In practical deployments, once a model has been trained, retraining or fine-tuning it is typically costly and may compromise system stability. This framework supports plug-and-play performance enhancement without altering existing model parameters, requiring only the additional training of lightweight semantic extraction and fusion modules. This proves particularly crucial for rapidly responding to novel threats and adapting to new traffic

Table 9
Malware classification results.

Dataset	Android				USTC-TFC				ustcMal			
	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
FlowPrint	0.3106 (±0.057)	0.7181 (±0.008)	0.2294 (±0.049)	0.3398 (±0.057)	0.5580 (±0.038)	0.8306 (±0.024)	0.3813 (±0.035)	0.4635 (±0.034)	0.4445 (±0.214)	0.8570 (±0.047)	0.4113 (±0.046)	0.5114 (±0.035)
FS-Net	0.8364 (±0.071)	0.8424 (±0.072)	0.8379 (±0.071)	0.8397 (±0.072)	0.6487 (±0.055)	0.6123 (±0.052)	0.5951 (±0.051)	0.5818 (±0.050)	0.8605 (±0.073)	0.8795 (±0.075)	0.8545 (±0.073)	0.8582 (±0.073)
Hi-Fi	0.7306 (±0.031)	0.7385 (±0.029)	0.7306 (±0.031)	0.7288 (±0.032)	0.5080 (±0.013)	0.5438 (±0.021)	0.5305 (±0.016)	0.4995 (±0.014)	0.8500 (±0.155)	0.8632 (±0.168)	0.8500 (±0.155)	0.8344 (±0.181)
ET-Bert	0.7485 (±0.091)	0.7526 (±0.085)	0.7499 (±0.086)	0.7458 (±0.090)	0.8232 (±0.035)	0.8649 (±0.023)	0.8551 (±0.021)	0.8538 (±0.019)	0.8762 (±0.015)	0.8870 (±0.027)	0.8499 (±0.027)	0.8426 (±0.029)
ET-Bert ef*	0.7489 (±0.061)	0.7570 (±0.053)	0.7508 (±0.056)	0.7464 (±0.060)	0.8215 (±0.029)	0.8687 (±0.019)	0.8560 (±0.016)	0.8549 (±0.015)	0.9120 (±0.011)	0.9102 (±0.013)	0.8980 (±0.013)	0.8898 (±0.015)
TrafficFormer	0.7667 (±0.047)	0.7783 (±0.040)	0.7667 (±0.047)	0.7665 (±0.044)	0.9095 (±0.010)	0.9328 (±0.006)	0.9310 (±0.007)	0.9302 (±0.006)	0.9524 (±0.006)	0.9561 (±0.007)	0.9524 (±0.006)	0.9522 (±0.006)
TrafficFormer ef	0.8520 (±0.045)	0.8550 (±0.044)	0.8520 (±0.045)	0.8516 (±0.045)	0.9226 (±0.013)	0.9429 (±0.008)	0.9407 (±0.010)	0.9403 (±0.010)	0.9488 (±0.010)	0.9531 (±0.010)	0.9488 (±0.010)	0.9485 (±0.010)
YaTC	0.8347 (±0.014)	0.8368 (±0.013)	0.8347 (±0.014)	0.8345 (±0.014)	0.9214 (±0.000)	0.9228 (±0.012)	0.8992 (±0.009)	0.9065 (±0.008)	0.9480 (±0.004)	0.9512 (±0.004)	0.9480 (±0.004)	0.9481 (±0.004)
YaTC ef	0.8333 (±0.026)	0.8401 (±0.024)	0.8333 (±0.026)	0.8337 (±0.025)	0.9228 (±0.003)	0.9356 (±0.005)	0.9067 (±0.008)	0.9138 (±0.009)	0.9448 (±0.011)	0.9490 (±0.010)	0.9448 (±0.011)	0.9450 (±0.010)
NetMamba	0.8013 (±0.050)	0.8089 (±0.042)	0.8013 (±0.050)	0.8020 (±0.048)	0.8632 (±0.013)	0.8912 (±0.010)	0.8657 (±0.021)	0.8738 (±0.016)	0.9432 (±0.005)	0.9468 (±0.005)	0.9432 (±0.005)	0.9430 (±0.004)
NetMamba ef	0.8186 (±0.014)	0.8211 (±0.012)	0.8186 (±0.014)	0.8191 (±0.014)	0.8612 (±0.025)	0.8998 (±0.020)	0.8727 (±0.023)	0.8817 (±0.021)	0.9444 (±0.011)	0.9476 (±0.009)	0.9444 (±0.011)	0.9445 (±0.011)

*“ef” indicates that the model has been enhanced using this framework.

Table 10
Use the framework after fine-tuning the model.

Dataset	CIC-IOT				UAV				ustcMal			
	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
ET-Bert	0.8435	0.8488	0.8347	0.8376	0.7386	0.7499	0.7354	0.7130	0.8564	0.8495	0.8047	0.7924
ET-Bert ef*	0.8478	0.8529	0.8400	0.8430	0.8295	0.8321	0.8115	0.8168	0.8911	0.8838	0.8715	0.8852
TrafficFormer	0.7840	0.7885	0.7840	0.7796	0.8387	0.8247	0.7979	0.8062	0.9500	0.9541	0.9500	0.9498
TrafficFormer ef	0.7880	0.7932	0.7880	0.7794	0.8571	0.8404	0.8294	0.8331	0.9500	0.9538	0.9500	0.9499
YaTC	0.8200	0.8195	0.8200	0.8165	0.8040	0.8045	0.8040	0.7989	0.9560	0.9582	0.9560	0.9554
YaTC ef	0.8320	0.8312	0.8320	0.8288	0.8120	0.8122	0.8120	0.8081	0.956	0.9574	0.9560	0.9556
NetMamba	0.8000	0.8003	0.8000	0.7993	0.7840	0.7819	0.7840	0.7823	0.9500	0.9536	0.9500	0.9488
NetMamba ef	0.8160	0.8157	0.8160	0.8156	0.8040	0.8030	0.8040	0.8029	0.9560	0.9605	0.9560	0.9550

*“ef” indicates that the model has been enhanced using this framework.

scenarios. For instance, within an already deployed malicious traffic detection system, integrating this framework can significantly improve recognition capabilities for unknown malicious traffic without necessitating a full model update, thereby greatly enhancing the system’s flexibility and maintainability. Notably, the lightweight design of this framework provides a viable deployment pathway for distributed scenarios such as fog computing and industrial network frontiers. Compared to fully federated approaches [51], this framework adopts a ‘central-edge’ collaborative strategy: PTM fine-tuning occurs on the central server, whilst the lightweight enhancement module can be deployed at edge nodes. This avoids communication delays caused by multiple rounds of global aggregation in federated learning whilst retaining the edge’s capacity for rapid adaptation to local traffic distributions. Consequently, it offers an alternative solution for resource-constrained distributed environments that balances efficiency and effectiveness.

5.4. The framework deep dive

In this section, an in-depth analysis of the key components of the framework is conducted in order to assess the specific impact of each component on classification performance. The ensuing discourse will concentrate on three aspects: feature interaction, statistical feature semantic extraction module, and multi-head self-attention gating mechanism. For the purpose of model evaluation, four representative datasets were selected for analysis: Android, APP, Tor and UAV. Among these, Android encompasses the widest range of software types; APP refers to encrypted traffic within VPN tunnels; Tor pertains to traffic within anonymous networks; and UAV represents traffic generated during users’ online activities, which is one of the most common traffic types. Through the evaluation of these datasets, it is possible to gain a comprehensive understanding of the roles and importance of each component within the framework.

Feature Interaction. The impact of interaction features (i.e., the Hadamard product of [SFSV] and [CLS]) on framework performance was evaluated. In order to control for variables, the framework without interaction features retained other hyperparameter values unchanged. The evaluation datasets for this study were selected to include App, Tor, and UAV.

As demonstrated in Fig. 6, a comparison of the classification F1 scores before and after the removal of feature intersection is provided. It is evident from the data that there has been a decline in the F1 score of TrafficFormer on the Tor dataset, with a decrease of 17.3%. A similar trend is observed in the F1 score of ET-Bert on the Tor dataset, which has shown a decline of 18.2%. Additionally, the F1 score of YaTC on the UAV dataset has decreased by 15.7%, while NetMamba’s F1 score on the Tor dataset has shown a decline of 8.9%. On the evaluation dataset, the classification performance of the framework generally decreased after removing interaction features, indicating that interaction information is effective for classification. This is due to the fact that the gating mechanism’s fundamental process is weighted averaging, which merely achieves feature selection but is incapable of expressing interactions between features. It has been demonstrated that feature interactions have the capacity to compensate for the linear limitations of the gating mechanism, thereby enhancing the model’s ability to generate non-linear combination features.

In general, the impact on YaTC and NetMamba was less significant in comparison to that experienced by ET-Bert and TrafficFormer. This is due to the fact that ET-BERT and TrafficFormer extract semantic information from byte sequences, a method that is analogous to the extraction method of our statistical feature semantic information extraction module. However, YaTC and NetMamba accept images as input, which differs significantly from the extraction method of our statistical feature semantic information extraction module. Consequently, the enhancement in performance demonstrated by ET-Bert and TrafficFormer

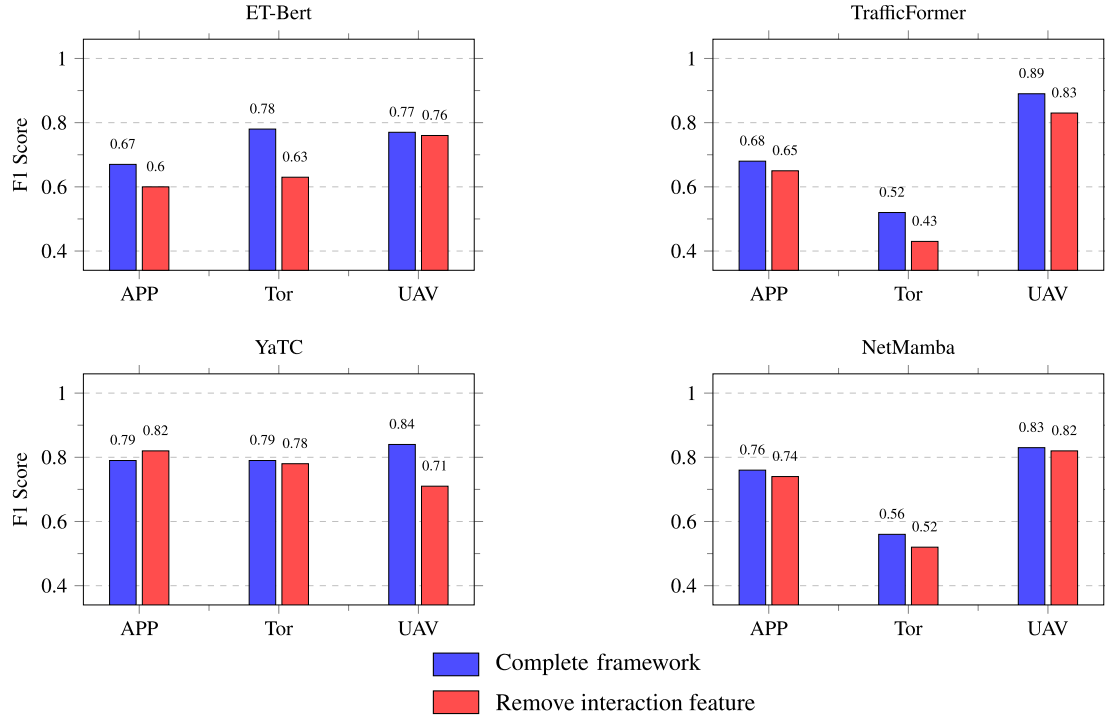


Fig. 6. The impact of interaction feature on the framework (F1).

Table 11

Performance comparison of activation functions.

Dataset	GeLU	LeakyReLU	Softplus	Sigmoid	tanh	Mish	ReLU
UAV	0.8917	0.8276	0.8319	0.8099	0.8888	0.8791	0.8611
USTC-TFC	0.9731	0.9748	0.9718	0.9719	0.9717	0.9674	0.9663
Tor	0.5204	0.4236	0.4389	0.4635	0.5273	0.4736	0.3533

through feature interaction vectors is more substantial than that observed in YaTC and NetMamba.

Performance Analysis of SFSEM Activation Functions. To validate the efficacy of SFSEM, using Trafficformer as the baseline model, the system compared GeLU with LeakyReLU, Softplus, Sigmoid, tanh, Mish, and ReLU across three representative datasets: UAV, USTC-TFC, and Tor. Experiments maintained consistent hyperparameters, with macro F1 score as the primary evaluation metric. Results are presented in Table 11.

The experimental results demonstrate that the performance of different activation functions exhibits significant dataset dependency, yet GeLU demonstrates optimal overall adaptability: on the UAV dataset, GeLU ranked first with a macro F1 score of 0.8917, achieving F1 improvements ranging from 0.0029 to 0.0818, reflecting its strong semantic extraction capability for user behaviour-based traffic statistical features; On the USTC-TFC dataset, all activation functions performed well overall. LeakyReLU achieved a marginally higher F1 score than GeLU, though the difference was only 0.0017. GeLU significantly outperformed Mish and ReLU, indicating its robustness in mapping statistical features of malware traffic. On the Tor dataset, tanh achieved a marginally higher F1 score than GeLU, yet GeLU still substantially outperformed the remaining activation functions, with performance gains ranging from 0.0568 to 0.1671. Particularly in scenarios like anonymous network traffic where semantic features are weakened, it demonstrated effective modelling capabilities for low-distinctiveness statistical features. GeLU delivered the most comprehensive performance across three distinct datasets, exhibiting no notable weaknesses. It effectively supported SFSEM in achieving precise mapping from statistical features to semantic space, thereby confirming its selection as

the activation function for the statistical feature semantic extraction module.

Comparison of Fusion Methods. To clarify the unique value of this paper's core innovation (SFSEM semantic alignment + GATING dynamic fusion), we compare it with five baseline fusion methods. All bypass the SFSEM and GATING mechanisms, achieving fusion between CLS and statistical feature vectors (SFV) solely through foundational techniques, defined as follows:

Statistical Features Only: No CLS is used; only the original 77-dimensional SFV is input to the classification head; **Direct Concatenation:** CLS and original SFV are concatenated directly along dimensions without semantic mapping or gating; **Simple Concatenation:** SFV is mapped to a 768-dimensional SFSV via SFSEM, then concatenated directly with CLS before input to the classification head; **Bilateral Pooling:** SFSV and CLS are separately mapped to 256 dimensions via distinct projection layers, second-order interaction features are computed through matrix multiplication, followed by dimensionality reduction and residual connection with CLS before input to the classification head; **Multi-layer Fusion MLP:** Concatenated SFSV and CLS undergo linear fusion through a three-layer MLP network.

We employed Trafficformer as the base model and evaluated the classification performance of six fusion methods across the UAV, USTC-TFC, and Tor datasets, using macro F1 score as the comparison metric. Results are presented in Table 12.

Experimental results demonstrate that our proposed fusion method achieves optimal or second-best performance across three representative datasets, comprehensively outperforming five baseline fusion approaches, thereby fully validating the method's effectiveness.

Lightweight Feature Evaluation. To validate the framework's lightweight nature, we conducted comparative analyses of parameter counts, floating-point operations per second (FLOPs), memory consumption, and inference latency on ET-Bert and Trafficformer. Both training and inference batch sizes were set to 32. Inference performance testing utilised 100,000 independent traffic data points to ensure the objectivity and representativeness of results. Results are presented in Table 13.

Experimental results demonstrate that the computational overhead of models after framework integration increases only marginally:

Table 12
Performance comparison of fusion methods.

Dataset	Ours	CLS*	Statistical Features Only	Direct Concatenation	Simple Concatenation	Bilateral Pooling	Multi-layer Fusion MLP
UAV	0.8917	0.8062	0.7164	0.8467	0.8422	0.8787	0.8680
USTC-TFC	0.9731	0.9686	0.6185	0.9701	0.9746	0.9624	0.9727
Tor	0.5204	0.4352	0.1538	0.4843	0.5069	0.4806	0.4414

*CLS refers to the model without the enhanced framework, i.e. Trafficformer.

Table 13
Comparison of computational resource expenditure before and after framework enhancement.

Model	Parameter Count (MB)	Floating-Point Operations (GFLOPs)	Training GPU Memory Requirement (MB)	Inference GPU Memory Requirement (MB)	Inference Duration (S)
ET-Bert	504.03	347.91	6541.77	1030.98	9418.81
ET-Bert ef*	552.12	347.94	6706.67	1116.42	9515.98
Trafficformer	504.03	869.75	16386.54	2895.57	7275.94
Trafficformer ef	545.42	869.78	16544.94	2937.00	7340.78

ast“ef” indicates that the model has been enhanced using this framework.

ET-Bert and Trafficformer saw parameter counts rise by 9.54% and 8.21% respectively, whilst single-pass FLOPs increased by merely 0.008% and 0.003%. Although model size expanded, computational complexity remained unchanged. Regarding GPU memory consumption, training-phase memory usage increased by merely 2.52% (ET-Bert) and 0.97% (Trafficformer), while inference-phase memory usage rose by 8.28% (ET-Bert) and 1.43% (Trafficformer), indicating no substantial increase in hardware resource requirements. Inference time increased by 1.02% (ET-Bert) and 0.89% (Trafficformer) compared to the original models, indicating minimal impact of the framework on model inference speed.

In summary, the proposed framework achieves a significant improvement in PTM classification performance at the cost of minimal computational overhead, demonstrating its lightweight nature and exceptional cost-effectiveness. The additional accuracy gains fully justify the modest computational expenditure.

GATING. As outlined in Section 4.5, the framework employs GATING to integrate [CLS] and [SFSV]. The following investigation will compare the performance differences between traditional gated networks and GATING. In order to achieve this objective, the GATING module is to be substituted with a conventional gated network, whilst ensuring that no alterations are made to other modules. Specifically, the multi-head self-attention mechanism was removed and the dimension of the gating parameters was adjusted to match the dimension of the vectors to be fused (i.e., [CLS] and [SFSV]). In order to control for variables, the interaction features were retained. Furthermore, all other model parameters remained constant. The Android, APP, and Tor datasets were selected for the evaluation process.

As Fig. 7 shows, GATING generally outperforms traditional gated networks. This may be because GATING has a self-attention mechanism that enables it to capture more complex interactions. Additionally, GATING’s gating parameter is set to one dimension, helping to maintain the semantic integrity of feature vectors.

Interpretability Analysis of the Dynamic Fusion Mechanism. To delve into the operational principles of the dynamic fusion mechanism and clarify the dependency relationships and variation patterns between SFSV and CLS under different traffic types, this section uses the Trafficformer model as an example. Box plots are employed to visualise the distribution of gating weights, thereby quantifying the differences in fusion proportions between the two feature types. Specifically, we extract the gating weights g (valued between 0 and 1) generated by the GATING module during framework operation. Their physical interpretation is clear: the closer g approaches 0, the greater the model’s reliance on SFSV; the closer g approaches 1, the greater the model’s reliance on CLS extracted by PTM.

As shown in Fig. 8, the distribution of gating weights across different traffic types exhibits marked divergence, closely aligning with patterns of classification performance enhancement: across the CIC-IoT, Android,

UAV, and Tor datasets, the distribution of gating weight g clusters near the zero value range, indicating that statistical features contribute more significantly to classification in these scenarios. Experimental results corroborate this pattern, with F1 scores exceeding 10% improvement across all four datasets—representing the most substantial performance gains among all tasks. Conversely, on the IDI, Service, APP, ustcMal, and USTC-TFC datasets, the distribution of g weight tends towards 1, indicating that model classification relies primarily on CLS semantic features. This aligns with the observed minimal change in classification performance before and after framework application on these datasets, as expected.

These results fully validate the adaptive adjustment capability of the dynamic fusion framework: it intelligently balances the contribution weights of statistical and semantic features based on the discriminative power of different traffic categories. When statistical features prove more conducive to enhancing classification accuracy, the gating mechanism automatically assigns them greater weight; conversely, when semantic features dominate, the framework preserves PTM’s semantic modelling advantages to achieve superior classification performance. This also substantiates the framework’s design rationality from an interpretability perspective.

Statistical Feature Importance Analysis. To quantify the contribution of each feature to the classification task, we adopt a permutation-based feature importance evaluation method and conduct inference experiments with Trafficformer as the base model. Specifically, inference is performed with the complete feature set, and the benchmark loss of the model is recorded; a single feature among the 77-dimensional features is randomly shuffled in turn while the other features remain unchanged; the model is retested with the shuffled feature set, and the loss variation is calculated. The loss variation is used as the metric for feature importance: the larger the loss variation, the higher the contribution of the corresponding feature to the model’s predictions, and the greater its importance. The above experiments are independently repeated on datasets from ten different scenarios. As shown in Fig. 9, the top ten features are selected in order of importance.

As shown in Fig. 9, feature importance exhibits pronounced task specificity, with discriminative features diverging across different scenarios. This finding provides bidirectional corroboration with the analysis of dynamic fusion mechanisms.

Specifically, the distribution of feature importance is not random but closely correlated with the intrinsic behavioural patterns of specific traffic types, providing a deep theoretical explanation for the bias in gating weights. In malware detection (e.g., Android), features such as idle time extremes and burst transmission rates become significantly more important. This is because malware often employs low-frequency, burst-based heartbeat or data exfiltration mechanisms to maintain stealth, manifesting as prolonged periods of silence followed by brief data explosions.

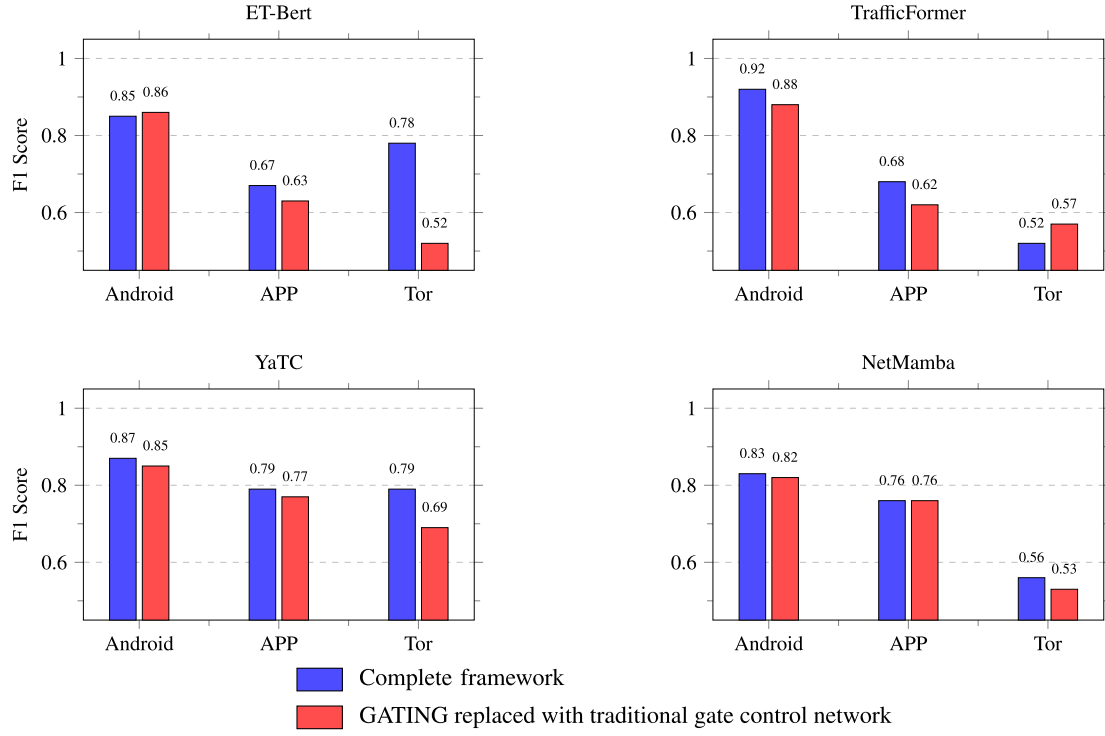


Fig. 7. The impact of GATING on the framework (F1).

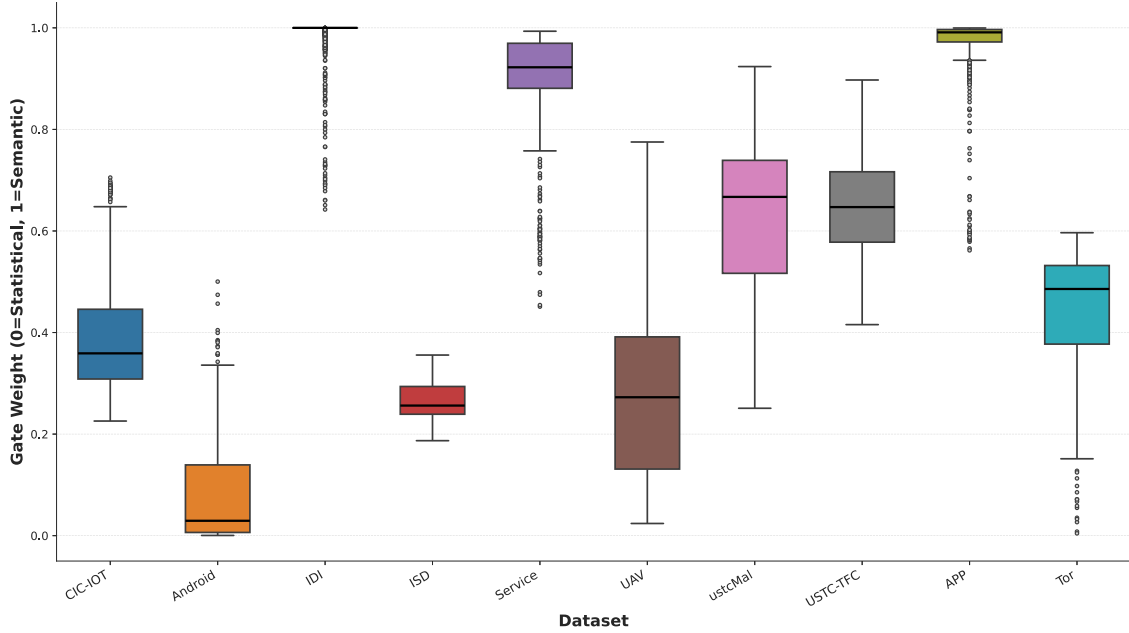


Fig. 8. Box-and-whisker plot of gate control weight distribution across different task types.

Consequently, statistical characteristics describing the rhythm of silence and bursts become critical behavioural fingerprints. In anonymous network traffic detection (e.g., Tor), feature importance centres on metrics like packet arrival intervals, total packet length, and packet count. This arises because while Tor's multi-layer encryption largely obscures payload semantic differences, the underlying asymmetry and rhythmic patterns from relay hops, link establishment, and bidirectional data transmission still leave side-channel traces in these macro-statistical characteristics. In IoT device identification (e.g., CIC-IOT), the importance of minimum packet interval and flow duration becomes prominent. This correlates closely with the behavioural characteristics of IoT

devices, which typically exhibit regular communication intervals and transient sessions. These findings on feature significance corroborate analyses of the dynamic fusion mechanism: in scenarios where statistical features exhibit pronounced discriminative power (e.g., Android, Tor, CIC-IOT, UAV), the distribution of gating weights clusters closer to the zero-value range. This indicates the framework's adaptive capability to identify and prioritise statistical features most reflective of a traffic class's intrinsic behaviour. Consequently, it achieves significant classification performance gains even when semantic features are weakened or obscured. This mechanism demonstrates the framework's capacity to dynamically adjust feature fusion strategies based on intrinsic traffic

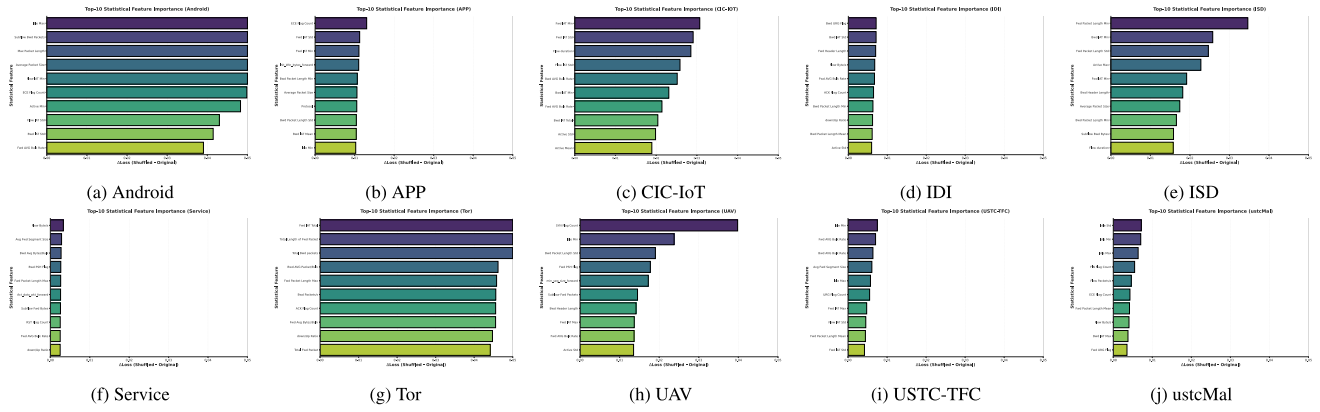


Fig. 9. Importance distribution of the top ten statistical features across ten datasets.

Table 14

Performance comparison of frameworks under different dimensional statistical characteristics.

Dataset	Android	APP	CIC-IoT	IDI	ISD	Service	Tor	UAV	USTC-TFC	ustcMal
Trafficformer /0*	0.8160	0.6878	0.7796	0.9222	0.9700	0.9802	0.4352	0.8062	0.9686	0.9498
Trafficformer /20	0.8333	0.6668	0.8634	0.9575	0.9800	0.9845	0.4241	0.8573	0.9740	0.9553
Trafficformer /40	0.8600	0.6769	0.8963	0.9524	0.9900	0.9811	0.5949	0.8476	0.9740	0.9458
Trafficformer /60	0.8059	0.6559	0.8665	0.9537	0.9800	0.9801	0.5839	0.8553	0.9718	0.9523
Trafficformer /77	0.9197	0.6759	0.8757	0.9342	0.9800	0.9719	0.5204	0.8917	0.9731	0.9452

* /n denotes extracting the first n-dimensional statistical features. When n is 0, it signifies that no framework is employed for augmentation, i.e., Trafficformer.

behavioural patterns, with decision-making grounded in interpretable traffic behavioural science principles.

Robustness Analysis of Feature Subset. To validate the robustness of statistical feature subsets, we designed the following experiment: Using the full set of 77 statistical features, we sequentially selected subsets comprising the top 20, top 40, and top 60 features (feature order detailed in Appendix A). Employing Trafficformer as the base model, we evaluated the framework's performance across ten classification tasks, with macro F1 score serving as the primary metric. Experimental results are presented in Table 14.

Experimental results demonstrate that the framework exhibits significant robustness to subsets of statistical features. Performance across all feature subsets shows no substantial decline, and in most tasks outperforms the baseline model without statistical features. For instance, on the CIC-IoT, the macro F1 scores for the 20-dimensional, 40-dimensional, and 60-dimensional subsets were 0.8634, 0.8963, and 0.8665 respectively, all significantly higher than the baseline model's 0.7796. On the IDI dataset, the macro F1 scores for all subsets exceeded 0.95, representing a stable improvement over the baseline model's 0.9222, thereby validating the effectiveness of the feature subsets.

Ablation Analysis of Duplicate Packet Removal Preprocessing. To quantify the impact of duplicate packet removal preprocessing on framework performance, we conducted comparative experiments with and without duplicate packet removal on six representative datasets—IDI, Service, ustcMal, USTC-TFC, APP, and Tor—using Trafficformer and ET-Bert as representative base models. Macro F1 score served as the primary evaluation metric. Experimental results are presented in Table 15.

Experimental results demonstrate that deduplication preprocessing of retransmitted packets yields overall positive gains in framework performance without introducing significant bias: in key scenarios reliant on statistical feature differentiation (Tor and Service datasets), model performance improvements following deduplication are particularly pronounced. ET-Bert's F1 score on the Tor dataset rose from 0.5975 to 0.7783 (a 30.3% increase), while Trafficformer's score on the same dataset improved from 0.4120 to 0.5204 (a 26.3% increase). This outcome corroborates the earlier hypothesis: retransmitted packets distort the accuracy of global statistical features in traffic. Their

Table 15

Performance comparison of frameworks with and without preprocessing for duplicate packet removal.

Model	IDI	Service	ustcMal	USTC-TFC	APP	Tor
ET-Bert (WoD)*	0.8832	0.8974	0.9249	0.9241	0.6933	0.5975
ET-Bert (WD) [†]	0.8772	0.9679	0.8700	0.9354	0.6664	0.7783
Trafficformer (WoD)	0.9478	0.9443	0.9597	0.9727	0.6345	0.4120
Trafficformer (WD)	0.9342	0.9719	0.9452	0.9731	0.6759	0.5204

* 'WD' denotes the extraction of statistical characteristics after removing duplicate retransmitted packets via the tshark tool.

[†] 'WoD' refers to the extraction of statistical features directly from raw traffic (including retransmitted packets).

removal enables these features to more faithfully reflect the intrinsic behaviour of traffic. This optimisation proves particularly crucial in scenarios where semantic distinctions are weak, such as encrypted traffic (Tor) and VPN service traffic (Service). Performance remained largely unchanged across other datasets after deduplication, indicating that this operation does not disrupt core traffic distribution characteristics and does not impair the framework's generalisation capabilities in practical deployment.

In summary, the design of retransmission packet deduplication as a preprocessing step is justified: this operation effectively removes network transmission noise (retransmitted packets caused by network fluctuations), significantly optimises performance in core scenarios where semantic feature differentiation is challenging, and introduces no significant bias in other scenarios.

6. Discussion

In this section, we will discuss some of the limitations of the framework.

Multi-stream Detection. The framework has not yet addressed the issue of multi-stream detection, it still only accepts input from a single stream. This restricts the model's classification performance in multi-stream scenarios. One way to mitigate this issue is to aggregate feature

vectors from multiple streams to comprehensively use multi-stream information (e.g. stream-to-trajectory mapping in the hierarchical deep learning framework [27]).

Insufficient Anti-confusion Ability. The performance of the framework relies to a certain extent on statistical characteristics extracted from network flows (such as packet length, packet interval, transmission rate, etc.). However, a primary objective of traffic obfuscation techniques is precisely to modify or conceal these side-channel characteristics to evade detection methods based on such features [52–54]. When traffic is heavily obfuscated, the statistical features exploited by the framework may become distorted or lose their discriminative power, consequently leading to a decline in classification performance. To enhance the framework's robustness in adversarial environments, future work may explore the following directions: - Designing higher-level statistical or sequence pattern features—such as packet counts over time [55]—that exhibit greater resilience to common obfuscation techniques (e.g., padding, temporal jitter, packet length normalisation); During training, simulate obfuscated traffic features through data augmentation or incorporate adversarial samples into the training process, enabling the model to learn more robust feature representations [56]; Extend frameworks from single-flow to coarser granularities (e.g., session-level, host-level), aggregating patterns across multiple flows to counteract the impact of locally obfuscated features within individual flows [57,58]. Furthermore, one may explore integrating bio-inspired approaches such as neuromorphic quantum adversarial learning [59] with quantum-augmented paradigms into this framework. By leveraging the low-power characteristics of neuromorphic computing and the parallel processing capabilities of quantum computing, one could construct feature representations and detection mechanisms that exhibit greater robustness against highly evasive traffic. Similarly, in privacy-preserving spatial range queries, researchers have employed obfuscated Bloom filters to conceal inclusion relationships, thereby enhancing query privacy and security [60]. This inspires future work to adapt such obfuscation techniques for strengthening traffic feature robustness against adversarial attacks.

Covert Attacks Driven by DoH and Similar Protocols. The DNS over HTTPS (DoH) protocol encapsulates DNS queries within HTTPS tunnels, rendering malicious traffic highly indistinguishable from benign web application traffic in terms of payload structure. Relying solely on semantic features proves insufficient for effective differentiation, presenting a critical challenge in the current field of encrypted traffic classification [61]. Nevertheless, DoH traffic still exhibits distinctive behavioural fingerprints in statistical characteristics such as time-series patterns, packet length distributions, query frequencies, and session durations [62]. Future research may draw upon advanced detection framework approaches (MTL-DoHTA [61]) and further explore the integration of federated AI authentication models with proactive defence strategies (PAFA-DoH [63]), whilst keeping abreast of the latest advancements in next-generation AI frameworks for DoH advanced threat detection and security enhancement [64]. This would involve designing and incorporating dedicated statistical feature sets tailored to DoH traffic, thereby significantly enhancing detection models' accuracy and robustness against such emerging covert attacks.

Deployment Challenges in Modern Large-Scale Network Environments. The framework design primarily targets offline or low-throughput traffic analysis scenarios, encountering real-time processing performance bottlenecks in modern high-bandwidth, low-latency network environments such as 5G core networks and cloud data centres. Specifically, statistical feature extraction, semantic mapping networks, and gated fusion mechanisms introduce additional computational overhead, potentially causing processing delays that fail to meet the line-rate processing requirements of 10-gigabit links [65]. Furthermore, the framework's reliance on flow statistics necessitates full computation only upon each network flow's conclusion, creating an inherent conflict

with streaming processing requirements [66]. Drawing inspiration from early classification approaches, a cascaded classification system may be devised [67]. The system pre-positions a lightweight model to perform rapid coarse classification using only the initial packets; samples with low confidence are then submitted to the model for fine-grained classification.

7. Conclusion

In this paper, we propose an enhanced framework for a PTM traffic detection model. The framework successfully overcomes the limitations of existing PTMs in processing network flow statistical features through an innovative semantic extraction network and gated fusion network. Experimental results demonstrate that the framework significantly enhances the classification performance of four mainstream PTMs across ten typical tasks without altering the model architecture. Its plug-in design enables rapid integration into existing security protection systems, providing an efficient, standardised solution for dynamic traffic monitoring in encrypted environments. Despite its strong experimental performance, this research faces notable limitations requiring attention: capturing cross-flow evolution patterns of complex attacks remains challenging; static statistical features may distort under adversarial conditions, introducing recognition risks; and balancing inference overhead for feature extraction and fusion in large-scale, high-speed networks persists. Future research may focus on specific domains addressing these challenges: multi-flow correlation modelling, robustness against traffic obfuscation, and real-time processing with hardware acceleration.

CRediT authorship contribution statement

Chaofan Zheng: Writing – original draft, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization; **Hailong Ma:** Writing – review & editing, Visualization, Validation, Supervision, Software; **Yanze Qu:** Supervision, Software, Resources, Project administration, Methodology; **Yiming Jiang:** Writing – review & editing, Visualization, Validation, Supervision, Software, Resources; **Wenbo Wang:** Writing – review & editing, Software, Resources, Project administration, Methodology, Investigation.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Feature selection

In order to ensure the framework's capacity for adaptation to the characteristics of the majority of datasets, the principle of feature selection was followed, with the objective of preserving features with broad applicability as much as possible, whilst excluding those with specific context dependencies, such as IP addresses and port numbers. In this study, we used the CICFlowMeter [34] project as a foundation for the extraction of 84-dimensional features, of which 77 were selected for further analysis. The detailed descriptions of these features can be found in Table A.1. The selection of these features was driven by their demonstrated capacity for effective discrimination and resilience across diverse network traffic types. This enhances the model's generalisation capability, leading to improved performance across various datasets.

Table A.1
Selected features.

Feature name	Description	Feature name	Description
Protocol	Network communication protocol	Max Packet Length	Maximum length of a packet
Flow Duration	Duration of the flow in Microsecond	Packet Length Mean	Mean length of a packet
Total Fwd Packet	Total packets in the forward direction	Packet Length Std	Standard deviation length of a packet
Total Bwd packets	Total packets in the backward direction	Packet Length Variance	Variance length of a packet
Total Length of Fwd Packet	Total size of packet in forward direction	FIN Flag Count	Number of packets with FIN
Total Length of Bwd Packet	Total size of packet in backward direction	SYN Flag Count	Number of packets with SYN
Fwd Packet Length Min	Minimum size of packet in forward direction	RST Flag Count	Number of packets with RST
Fwd Packet Length Max	Maximum size of packet in forward direction	PSH Flag Count	Number of packets with PUSH
Fwd Packet Length Mean	Mean size of packet in forward direction	ACK Flag Count	Number of packets with ACK
Fwd Packet Length Std	Standard deviation size of packet in forward direction	URG Flag Count	Number of packets with URG
Bwd Packet Length Min	Minimum size of packet in backward direction	CWR Flag Count	Number of packets with CWR
Bwd Packet Length Max	Maximum size of packet in backward direction	ECE Flag Count	Number of packets with ECE
Bwd Packet Length Mean	Mean size of packet in backward direction	down/Up Ratio	Download and upload ratio
Bwd Packet Length Std	Standard deviation size of packet in backward direction	Average Packet Size	Average size of packet
Flow Byte/s	Number of flow packets per second	Avg Fwd Segment Size	Average size observed in the forward direction
Flow Packets/s	Number of flow bytes per second	AVG Bwd Segment Size	Average number of bytes bulk rate in the backward direction
Flow IAT Mean	Mean time between two packets sent in the flow	Fwd Avg Bytes/Bulk	Average number of bytes bulk rate in the forward direction
Flow IAT Std	Standard deviation time between two packets sent in the flow	Fwd AVG Packet/Bulk	Average number of packets bulk rate in the forward direction
Flow IAT Max	Maximum time between two packets sent in the flow	Fwd AVG Bulk Rate	Average number of bulk rate in the forward direction
Flow IAT Min	Minimum time between two packets sent in the flow	Bwd Avg Bytes/Bulk	Average number of bytes bulk rate in the backward direction
Fwd IAT Min	Minimum time between two packets sent in the forward direction	Bwd AVG Packet/Bulk	Average number of packets bulk rate in the backward direction
Fwd IAT Max	Maximum time between two packets sent in the forward direction	Bwd AVG Bulk Rate	Average number of bulk rate in the backward direction
Fwd IAT Mean	Mean time between two packets sent in the forward direction	Subflow Fwd Packets	The average number of packets in a sub flow in the forward direction
Fwd IAT Std	Standard deviation time between two packets sent in the forward direction	Subflow Fwd Bytes	The average number of bytes in a sub flow in the forward direction
Fwd IAT Total	Total time between two packets sent in the forward direction	Subflow Bwd Packets	The average number of packets in a sub flow in the backward direction
Bwd IAT Min	Minimum time between two packets sent in the backward direction	Subflow Bwd Bytes	The average number of bytes in a sub flow in the backward direction
Bwd IAT Max	Maximum time between two packets sent in the backward direction	Init_Win_bytes_forward	The total number of bytes sent in initial window in the forward direction
Bwd IAT Mean	Mean time between two packets sent in the backward direction	Init_Win_bytes_backward	The total number of bytes sent in initial window in the backward direction
Bwd IAT Std	Standard deviation time between two packets sent in the backward direction	Act_data_pkt_forward	Count of packets with at least 1 byte of TCP data payload in the forward direction
Bwd IAT Total	Total time between two packets sent in the backward direction	min_seg_size_forward	Minimum segment size observed in the forward direction
Fwd PSH flag	Number of times the PSH flag was set in packets travelling in the forward direction (0 for UDP)	Active Min	Minimum time a flow was active before becoming idle
Bwd PSH Flag	Number of times the PSH flag was set in packets travelling in the backward direction (0 for UDP)	Active Mean	Mean time a flow was active before becoming idle
Fwd URG Flag	Number of times the URG flag was set in packets travelling in the forward direction (0 for UDP)	Active Max	Maximum time a flow was active before becoming idle
Bwd URG Flag	Number of times the URG flag was set in packets travelling in the backward direction (0 for UDP)	Active Std	Standard deviation time a flow was active before becoming idle
Fwd Header Length	Total bytes used for headers in the forward direction	Idle Min	Minimum time a flow was idle before becoming active
Bwd Header Length	Total bytes used for headers in the backward direction	Idle Mean	Mean time a flow was idle before becoming active
FWD Packets/s	Number of forward packets per second	Idle Max	Maximum time a flow was idle before becoming active
Bwd Packets/s	Number of backward packets per second	Idle Std	Standard deviation time a flow was idle before becoming active
Min Packet Length	Minimum length of a packet		

Appendix B. Abbreviation reference table

For the convenience of quick reference, we provide a glossary of abbreviations used in this text, as shown in [Table B.1](#).

Table B.1
List of abbreviations.

Abbreviation	Full Form	Abbreviation	Full Form
PTM	Pre-trained Model	IDS	Intrusion detection system
HTTPS	Hypertext Transfer Protocol Secure	DoS	Denial of Service
CLS	Classification Token	MLP	Multilayer Perceptron
SFV	Statistical Feature Vectors	SFSV	Statistical Feature Semantic Vector
CFM	CICFlowMeter	GATING	Multi-head Self-attention Gating Mechanism
SFSEM	Statistical Feature Semantic Extraction Module	FFM	Feature Fusion Module
DWIV	Dynamic Weighted Interaction Vector	VPN	Virtual Private Network
IoT	Internet of Things	AC	Accuracy
PR	Precision	RC	Recall
F1	F1 Score		

Appendix C. Quantitative analysis of information loss due to input truncation

To address the lack of empirical evidence regarding motivation, we conducted a quantitative analysis of information loss caused by input truncation to validate the significance of the existing constraint on PTM input length.

All four baseline models (ET-Bert, YaTC, TrafficFormer, NetMamba) employ a ‘first five packets’ input cap (see Table 5), meaning models can only capture traffic information within this range, with excess data entirely discarded due to truncation. Based on this, we statistically analysed network flow features across ten classification task datasets, defining two core quantitative metrics: 1) Proportion of flows exceeding 5 packets: The ratio of network flows with packet counts exceeding 5 to the total flows in the dataset, quantifying the scope of input truncation impact; 2) Average packet count for flows exceeding 5 packets: Calculated solely from flows with packet counts exceeding 5, where the difference from 5 directly represents the packet volume inevitably lost by the model due to input truncation.

The quantitative statistical results are presented in Table C.1. The data reveal that over 50% of the packet streams in five datasets—CIC-IoT, Android, Tor, UAV, and others—exceeded five packets. Among these, the Android dataset exhibited the highest proportion (63.15%), while the ISD dataset had the lowest but still reached 4.25%. This indicates that over half of the datasets experienced information loss due to input truncation in their network streams, with the impact being widespread. The average packet count for all datasets’ over-5-packet flows exceeded 20, with the Tor dataset recording the highest (67,438.43) and the IDI dataset the lowest (20.21). . Calculating the loss ratio reveals that information loss exceeds 75% across all datasets, with Tor, ISD, and UAV datasets approaching 100% loss. Although some redundant information may exist within network flows, such a high proportion of discarded packets results in the loss of critical global features like packet length distribution and transmission rates. This further validates the necessity of incorporating statistical features to supplement semantic characteristics.

Table C.1
Quantitative statistics of information loss due to truncation in network flow inputs across datasets.

Statistical Indicator	IDI	Service/APP	ustcMal	USTC-TFC	Tor	CIC-IoT	Android	ISD	UAV
Excess ratio (%) [*]	55.47	15.22	51.22	38.13	61.00	42.16	63.15	4.25	57.12
Average number of packets [†]	20.21	2103.23	25.37	25.28	67438.43	773.94	60.18	58645.78	3470.47

^{*}The proportion of network flows where the number of packets exceeds five. [†]Refers to the average total number of packets in the excess flow.

References

- [1] Google transparency report, 2022. Accessed: 2025-05-8, <https://transparencyreport.google.com/>.
- [2] ThreatLabz 2023 state of encrypted attacks report - zscaler, 2023. Accessed: 2025-05-8, <https://www.zscaler.com/resources/2023-threatlabz-state-of-encrypted-attacks-report>.
- [3] H.Y. He, Z.G. Yang, X.N. Chen, PERT: payload encoding representation from transformer for encrypted traffic classification, in: 2020 ITU Kaleidoscope: Industry-Driven Digital Transformation (ITU K), IEEE, 2020, pp. 1–8.
- [4] W. Xiong, J. Liu, I. Molybog, H. Zhang, P. Bhargava, R. Hou, L. Martin, R. Rungta, K.A. Sankaraman, B. Oguz, et al., Effective long-context scaling of foundation models, (2023), arXiv preprint arXiv:2309.16039.
- [5] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, J. Yu, ET-BERT: a contextualized datagram representation with pre-training transformers for encrypted traffic classification, in: Proceedings of the ACM Web Conference 2022, (2022), pp. 633–642.
- [6] G. Zhou, X. Guo, Z. Liu, T. Li, Q. Li, K. Xu, Trafficformer: an efficient pre-trained model for traffic data, in: 2025 IEEE Symposium on Security and Privacy (SP), IEEE, 2025, pp. 1844–1860.
- [7] R. Zhao, M. Zhan, X. Deng, Y. Wang, Y. Wang, G. Gui, Z. Xue, Yet another traffic classifier: a masked autoencoder based traffic transformer with multi-level flow representation, Proc. AAAI Conf. Artif. Intell. 37 (2023) 5420–5427.
- [8] T. Wang, X. Xie, W. Wang, C. Wang, Y. Zhao, Y. Cui, NetMamba: efficient network traffic classification via pre-training unidirectional Mamba, in: 2024 IEEE 32nd International Conference on Network Protocols (ICNP), IEEE, 2024, pp. 1–11.
- [9] H. Lu, J. Liu, J. Peng, J. Lu, Adversarial attacks based on time-series features for traffic detection, Comput. Secur. 148 (2025) 104175.
- [10] S. Acharya, B.N. Mills, M. Abliz, T. Znati, J. Wang, Z. Ge, A.G. Greenberg, OPTWALL: a hierarchical traffic-aware firewall, in: NDSS, 2007.
- [11] A.Y. Botvinko, K.E. Samouylov, Evaluation of firewall performance metrics with ranging the rules for Poisson incoming packet flow and exponential filtering time, Discrete Contin. Models Appl. Comput. Sci. 31 (4) (2023) 345–358. <https://journals.rudn.ru/miph/article/view/37515>.
- [12] H. Woisetschlager, A. Erben, S. Wang, R. Mayer, H.-A. Jacobsen, Federated fine-tuning of llms on the very edge: the good, the bad, the ugly, in: Proceedings of the Eighth Workshop on Data Management for End-to-End Machine Learning, 2024, pp. 39–50.
- [13] S. Cai, X. Liu, J. Yuan, Q. Zhou, Prompt-ladder: memory-efficient prompt tuning for vision-language models on edge devices, Pattern Recognit. 163 (2025) 111460.
- [14] A.W. Moore, D. Zuev, Internet traffic classification using Bayesian analysis techniques, in: Proceedings of the 2005 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, 2005, pp. 50–60.
- [15] L. Bernaille, R. Teixeira, I. Akodkenou, A. Soule, K. Salamatian, Traffic classification on the fly, ACM SIGCOMM Comput. Commun. Rev. 36 (2) (2006) 23–26.
- [16] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, K. Wehrle, Website fingerprinting at internet scale, in: NDSS, 1, (2016), pp. 23477.
- [17] K. Al-Naami, S. Chandra, A. Mustafa, L. Khan, Z. Lin, K. Hamlen, B. Thuraisingham, Adaptive encrypted traffic fingerprinting with bi-directional dependence, in: Proceedings of the 32nd Annual Conference on Computer Security Applications, 2016, pp. 177–188.
- [18] V.F. Taylor, R. Spolaor, M. Conti, I. Martinovic, Robust smartphone app identification via encrypted network traffic analysis, IEEE Trans. Inf. Forensics Secur. 13 (1) (2017) 63–78.
- [19] T. van Ede, R. Bortolameotti, A. Continella, J. Ren, D.J. Dubois, M. Lindorfer, D.R. Choffnes, M. van Steen, A. Peter, FlowPrint: semi-supervised mobile-app fingerprinting on encrypted network traffic, in: 27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23–26, 2020, The Internet Society, 2020.
- [20] Z. Sabahi-Kaviani, F. Ghassemi, An encrypted traffic classifier via combination of deep learning and automata learning, Soft Comput. 28 (23) (2024) 13443–13460.
- [21] V. Rimmer, D. Preuveneers, M. Juarez, T. Van Goethem, W. Joosen, Automated website fingerprinting through deep learning, (2017), arXiv preprint arXiv:1708.06376.
- [22] P. Sirinam, M. Imani, M. Juarez, M. Wright, Deep fingerprinting: undermining website fingerprinting defenses with deep learning, in: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, 2018, pp. 1928–1943.
- [23] T. Shapira, Y. Shavitt, FlowPic: encrypted internet traffic classification is as easy as image recognition, in: IEEE INFOCOM 2019-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), IEEE, 2019, pp. 680–687.
- [24] Y. Mirsky, T. Doitshman, Y. Elovici, A. Shabtai, Kitsune: an ensemble of autoencoders for online network intrusion detection, (2018), arXiv preprint arXiv:1802.09089.
- [25] M. Shen, J. Zhang, L. Zhu, K. Xu, X. Du, Accurate decentralized application identification via encrypted traffic analysis using graph neural networks, IEEE Trans. Inf. Forensics Secur. 16 (2021) 2367–2380.
- [26] C. Liu, L. He, G. Xiong, Z. Cao, Z. Li, FS-Net: a flow sequence network for encrypted traffic classification, in: IEEE INFOCOM 2019-IEEE Conference on Computer Communications, IEEE, 2019, pp. 1171–1179.
- [27] J. Qu, X. Ma, J. Li, X. Luo, L. Xue, J. Zhang, Z. Li, L. Feng, X. Guan, An (input-agnostic) hierarchical deep learning framework for traffic fingerprinting, in: 32nd USENIX Security Symposium (USENIX Security 23), 2023, pp. 589–606.
- [28] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), 2019, pp. 4171–4186.
- [29] M.E. Peters, W. Ammar, C. Bhagavatula, R. Power, Semi-supervised sequence tagging with bidirectional language models, (2017), arXiv preprint arXiv:1705.00108.
- [30] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever, et al., Improving language understanding by generative pre-training, Technical report, OpenAI (2018). https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.
- [31] Z. Han, F. Yang, J. Huang, C. Zhang, J. Yao, Multimodal dynamics: dynamical fusion for trustworthy multimodal classification, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 20707–20717.
- [32] Z. Xue, R. Marculescu, Dynamic multimodal fusion, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops, 2023, pp. 2575–2584.
- [33] W. Dong, H. Zhu, S. Lin, X. Luo, Y. Shen, X. Liu, J. Zhang, G. Guo, B. Zhang, Fusion-Mamba for cross-modality object detection, arXiv e-prints (2024) arXiv:2404.09146.
- [34] I. Sharafaldin, A.H. Lashkari, A.A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: P. Mori, S. Furnell, O. Camp (Eds.), Proceedings of the 4th International Conference on Information Systems Security and Privacy, ICISPP 2018, Funchal, Madeira - Portugal, January 22–24, 2018, SciTePress, 2018, pp. 108–116.
- [35] W. Wang, M. Zhu, X. Zeng, X. Ye, Y. Sheng, Malware traffic classification using convolutional neural network for representation learning, in: 2017 International Conference on Information Networking (ICOIN), 2017, pp. 712–717.
- [36] Wireshark Foundation, Wireshark, 2025, ([Computer software]). Version 4.2.3, <https://github.com/wireshark/wireshark>.
- [37] M. Sarhan, S. Layeghy, M. Portmann, Evaluating standard feature sets towards increased generalisability and explainability of ML-based network intrusion detection, Big Data Res. 30 (2022) 100359. <https://doi.org/10.1016/j.bdr.2022.100359>
- [38] X. Glorot, A. Bordes, Y. Bengio, Deep sparse rectifier neural networks, in: G.J. Gordon, D.B. Dunson, M. Dudik (Eds.), Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, AISTATS 2011, Fort Lauderdale, USA, April 11–13, 2011, Vol. 15 of JMLR Proceedings, JMLR.org, 2011, pp. 315–323.
- [39] A.L. Maas, Rectifier nonlinearities improve neural network acoustic models, 2013. <https://api.semanticscholar.org/CorpusID:16489696>.
- [40] D. Misra, Mish: a self regularized non-monotonic activation function, in: 31st British Machine Vision Conference 2020, BMVC 2020, Virtual Event, UK, September 7–10, 2020, BMVA Press, 2020. <https://www.bmvc2020-conference.com/assets/papers/0928.pdf>.
- [41] D. Hendrycks, K. Gimpel, Gaussian error linear units (GELUs), arXiv e-prints (2016) arXiv:1606.08415.
- [42] M. Farhadul Islam, M. Tanzim Reza, M. Arafat Manab, M. Rakibul Hasan Mahin, S. Zabeen, J. Noor, Spatially optimized compact deep metric learning model for similarity search, arXiv e-prints (2024) arXiv:2404.06593.
- [43] S. Dadkhah, H. Mahdikhani, P.K. Danso, A. Zohourian, K.A. Truong, A.A. Ghorbani, Towards the development of a realistic multidimensional IoT profiling dataset, in: 19th Annual International Conference on Privacy, Security & Trust, PST 2022, Fredericton, NB, Canada, August 22–24, 2022, IEEE, 2022, pp. 1–11. <https://doi.org/10.1109/PST55820.2022.9851966>
- [44] M. Miettinen, S. Marchal, I. Hafeez, N. Asokan, A.-R. Sadeghi, S. Tarkoma, IoT SENTINEL: automated device-type identification for security enforcement in IoT, in: 2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS), 2017, pp. 2177–2184. <https://doi.org/10.1109/ICDCS.2017.283>
- [45] ETF IoT botnet dataset, 2021. Accessed: 2025-03-16, <https://data.mendeley.com/datasets/nbs6kvx6n/1>.
- [46] G. Draper-Gil, A.H. Lashkari, M.S.I. Mamun, A.A. Ghorbani, Characterization of encrypted and VPN traffic using time-related features, in: O. Camp, S. Furnell, P. Mori (Eds.), Proceedings of the 2nd International Conference on Information Systems Security and Privacy, ICISPP 2016, Rome, Italy, February 19–21, 2016, SciTePress, 2016, pp. 407–414.

- [47] A.H. Lashkari, G. Draper-Gil, M.S.I. Mamun, A.A. Ghorbani, Characterization of tor traffic using time based features, in: P. Mori, S. Furnell, O. Camp (Eds.), *Proceedings of the 3rd International Conference on Information Systems Security and Privacy, ICISSP 2017, Porto, Portugal, February 19-21, 2017*, SciTePress, 2017, pp. 253–262.
- [48] V. Labayen, E. Magaña, D. Morato, M. Izal, Online classification of user activities using machine learning on network traffic, *Comput. Netw.* 181 (2020) 107557. <https://doi.org/10.1016/j.comnet.2020.107557>
- [49] A.H. Lashkari, A.K. Andi Fitriah, H. Gonzalez, K.F. Mbah, A.G. Ali, Towards a network-based framework for android malware detection and characterization, in: *2017 15th Annual Conference on Privacy, Security and Trust (PST)*, 2017, pp. 233–23309.
- [50] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, *arXiv e-prints* (2017) [arXiv:1711.05101](https://arxiv.org/abs/1711.05101).
- [51] B. Ali, On the fog's frontline: a federated machine learning approach for industrial network threat detection and intrusion prevention, *J. Cybersecur.* 11 (1) (2025). <https://doi.org/10.1093/CYBSEC/TYAF041>
- [52] J. Huang, W. Liu, G. Liu, B. Gao, F. Nie, WF-A2D: enhancing privacy with asymmetric adversarial defense against website fingerprinting, *IEEE Trans. Inf. Forensics Secur.* 20 (2025) 4739–4754.
- [53] J. Gong, W. Zhang, C. Zhang, T. Wang, Surakav: generating realistic traces for a strong website fingerprinting defense, in: *2022 IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 1558–1573.
- [54] M. Shen, K. Ji, J. Wu, Q. Li, X. Kong, K. Xu, L. Zhu, Real-time website fingerprinting defense via traffic cluster anonymization, in: *2024 IEEE Symposium on Security and Privacy (SP)*, 2024, pp. 3238–3256.
- [55] X. Deng, R. Zhao, Y. Wang, M. Zhan, Z. Xue, Y. Wang, CountMamba: a generalized website fingerprinting attack via coarse-grained representation and fine-grained prediction, in: *2025 IEEE Symposium on Security and Privacy (SP)*, 2025, pp. 1419–1437.
- [56] Y. Merkli, R. Meier, M. Strohmeier, V. Lenders, Defeating and improving network flow classifiers through adversarial machine learning, in: *2024 16th International Conference on Cyber Conflict: Over the Horizon (CyCon)*, IEEE, 2024, pp. 103–121.
- [57] J. Chen, P. Wang, J. Liu, Y. Qian, Label relation graphs enhanced hierarchical residual network for hierarchical multi-granularity classification, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 4858–4867.
- [58] M. Shen, K. Ji, Z. Gao, Q. Li, L. Zhu, K. Xu, Subverting website fingerprinting defenses with robust traffic representation, in: *32nd USENIX Security Symposium (USENIX Security 23)*, USENIX Association, Anaheim, CA, 2023, pp. 607–624.
- [59] B. Ali, G. Chen, Neuromorphic quantum adversarial learning (NQAL): a bio-inspired paradigm for DNS over HTTPS threat detection, *EURASIP J. Inf. Secur.* 2025 (1) (2025) 35. <https://doi.org/10.1186/S13635-025-00215-5>
- [60] Y. Miao, Y. Yang, X. Li, Z. Liu, H. Li, K.R. Choo, R.H. Deng, Efficient privacy-preserving spatial range query over outsourced encrypted data, *IEEE Trans. Inf. Forensics Secur.* 18 (2023) 3921–3933. <https://doi.org/10.1109/TIFS.2023.3288453>
- [61] W.K. Jung, B.I. Kwak, MTL-DoHTA: multi-task learning-based DNS over HTTPS traffic analysis for enhanced network security, *Sensors* 25 (4) (2025) 993.
- [62] T. Dahanayaka, Z. Wang, G. Jourjon, S. Seneviratne, Inline traffic analysis attacks on DNS over HTTPS, in: *2022 IEEE 47th Conference on Local Computer Networks (LCN)*, IEEE, 2022, pp. 132–139.
- [63] B. Ali, G. Chen, Proactive and privacy-preserving defense for DNS over HTTPS via federated AI attestation (PAFA-DoH), *Neural Netw.* 196 (2026) 108343. <https://doi.org/10.1016/J.NEUNET.2025.108343>
- [64] B. Ali, G. Chen, Next-generation AI for advanced threat detection and security enhancement in DNS over HTTPS, *J. Netw. Comput. Appl.* 244 (2025) 104326. <https://doi.org/10.1016/J.JNCA.2025.104326>
- [65] S. Daum, T. Shapira, A. Bremner-Barr, D. Hay, Non-uniformity is all you need: efficient and timely encrypted traffic classification with echo, in: *2025 IEEE 33rd International Conference on Network Protocols (ICNP)*, IEEE, 2025, pp. 1–12.
- [66] S. Cui, X. Han, D. Han, Z. Wang, W. Wang, B. Jiang, B. Liu, Z. Lu, FG-SAT: efficient flow graph for encrypted traffic classification under environment shifts, *IEEE Trans. Inf. Forensics Secur.* 20 (2025) 5326–5339.
- [67] H. Mei, G. Cheng, Y. Yuan, High precision and efficient anonymous traffic classification in the real-world, *IEEE Trans. Netw.* 33 (2025) 966–981.